

Algorithm Specification and Supporting Documentation

WEAVER: A Lattice Based Key Encapsulation Mechanism

Long Chen¹, Zidi Zhuang¹, Tanping Zhou², Jianhua Hou³, Si Gao¹, Yuyang Xiao¹,
Yingjie Hao¹, Shuo Chen², and Ruiqi Mi¹

¹Institute of Software, Chinese Academy of Sciences, China

²Engineering University of PAP, China

³Information Engineering University, China

June 30, 2026

Abstract

This document presents the specification, security analysis, and performance evaluation of WEAVER, a lattice-based key encapsulation mechanism (KEM) submitted to the New Generation Commercial Cryptography (NGCC) algorithm standardization process. Our design is motivated by a critical challenge in modern lattice-based cryptography: while recent dual attacks have revealed that standard schemes like Kyber-512 exhibit narrower security margins than originally estimated, strengthening their security is notoriously difficult. Standard parameter-tuning approaches inevitably face a trilemma; aggressively adjusting compression or noise parameters to boost lattice hardness either causes the decryption failure rate (DFR) to spike unacceptably, or breaks the Number Theoretic Transform (NTT) friendliness, severely degrading concrete performance.

To break this trilemma, WEAVER introduces a balanced design paradigm that harmonizes size, performance, and security redundancy through two core innovations. First, we propose a creative hierarchical multi-layer encoding framework combining BCH codes with a 4-fold repetition structure. This coding strategy allows us to pack a larger number of message bits alongside robust error-correcting codes within a highly compressed ciphertext space. Second, to preserve high-speed polynomial arithmetic, we utilize a prime-modulus learning-with-rounding (LWR) structure. We resolve the incompatibility between prime moduli and LWR security reductions by introducing Randomized Lifting to maintain public-key uniformity, and formulate the Shifted-MLWR problem to justify our “embed-then-round” ciphertext structure. This allows WEAVER to fully leverage highly optimized, NTT-friendly prime-modulus arithmetic while achieving the transmission compactness of LWR. Overall, WEAVER demonstrates that by co-designing algebraic lattice structures with multi-layer error-correcting codes, one can successfully break the parameter-tuning deadlock and achieve a balanced trade-off between bandwidth, execution speed, and robust security redundancy.

Contents

1	Introduction	4
1.1	Main Features and Advantages	4
2	Algorithm Description	6
2.1	Preliminaries and Notation	6
2.2	Data Encoding and Serialization	8
2.3	Algorithm Specification	9
2.3.1	Specification of IND-CPA-secure PKE	9
2.3.2	Specification of IND-CCA-secure KEM	14
2.4	Symmetric Primitives and Quantum Security Levels	15
2.5	Parameter Sets	16
3	Design Rationale	18
3.1	Underlying Hard Problems	18
3.2	Message Encoding Strategy and Error Correction	20
3.3	Parameter Selection Strategy	21
3.3.1	Algebraic Parameters (n, q)	21
3.3.2	Security Scaling Parameters (k, η_1, η_2)	22
3.3.3	Bandwidth Compression Parameters (d_t, d_u, d_v)	22
3.3.4	Error Correction Parameters (BCH and Repetition)	22
3.4	Decryption Failure Analysis	22
3.4.1	Layered Decoding Thresholds	23
3.4.2	Single Bit Failure Rate	23
3.4.3	Independence Assumption and Overall Decryption Failure Rate	25
3.5	Rationale for Symmetric Primitives	26
3.5.1	Implicit Rejection FO Transform	26
4	Security Statement and Analysis	27
4.1	Theoretical Security	27
4.1.1	Hardness Assumptions	27
4.1.2	From IND-CPA PKE to IND-CCA KEM	30
4.2	Concrete Security Estimates	32
4.3	Side-Channel Resistance and Implementation Security	36
4.4	Other Security Properties	37
5	Performance Evaluation and Implementation Analysis	38
5.1	Evaluation Methodology and Platforms	38
5.2	Software Performance on x86 Processors	39
5.2.1	Weaver: Reference vs. AVX2	39
5.2.2	Comparison with ML-KEM and Rationale	39
5.3	Performance on Embedded Systems	41
5.3.1	Experimental Setup	41
5.3.2	Benchmarking Results	42
5.4	Optimization Techniques	42
5.4.1	Vectorization with AVX2 (x86_64)	43
5.5	Transmission and Storage Overhead	43
6	Features Statement	44
6.1	Innovation: No-Compromise Security and Efficiency	44
6.2	Compatibility and Real-World Applicability	45

A	From MLWE to Shifted-MLWR	49
A.1	Definitions and known tools from Liu–Wang.	49
A.2	Product shifted rounding map.	49
A.3	Main Reduction Theorem	51
A.4	Scope and Justification of the Shifted-MLWR Assumption	55

1 Introduction

This document provides the complete specification, security analysis, and performance evaluation for WEAVER, a lattice-based Key Encapsulation Mechanism (KEM) submitted to the New Generation Commercial Cryptography (NGCC) algorithm call. WEAVER is designed to achieve a carefully optimized balance between high computational efficiency, extremely compact transmission sizes, and robust post-quantum security with ample safety margins.

The standardization of ML-KEM (Kyber) [11] was a milestone in post-quantum cryptography. However, recent advancements in lattice cryptanalysis have significantly eroded its concrete security margins. Under the conservative Core-SVP cost model [7], the best known dual attacks—most notably the MATZOV dual attack [35] and its refined, assumption-free variant by Carrier et al. [12]—have shown that the actual hardness of the underlying MLWE instances in Kyber falls below their originally intended security targets.

Specifically, as illustrated in recent cryptanalytic evaluations, the classical and quantum security of Kyber-512 under the dual attack [35] drops to 115.5 bits and 103.2 bits, respectively. Even under the refined dual attack [12], Kyber-512’s classical security is estimated at 118.5 bits. Under these conservative Core-SVP estimates, Kyber-512 appears to provide a narrower concrete classical margin than the nominal 128-bit target required by the NGCC evaluation framework. Similarly, for the highest security level, Kyber-1024’s quantum security is degraded to 213.0 bits under [12], leaving a substantial gap from the expected 256-bit quantum security target.

This shrinking security margin presents a critical challenge for the transition to post-quantum standards, especially under the NGCC guidelines which explicitly mandate that candidate algorithms must provide sufficient security redundancy to mitigate the risks of future cryptanalytic breakthroughs.

To resolve this tension, WEAVER introduces a paradigm shift in how ciphertext compression, noise, and decryption failures are balanced. We adopt a design philosophy of Precise Noise Engineering: rather than conservatively bounding noise to avoid decryption failures (which forces the use of larger, less secure lattice parameters to maintain correctness), WEAVER completely eliminates explicitly sampled error vectors and aggressively compresses the transmitted elements. The resulting heavy rounding noise is treated as an active cryptographic hardness source (Shifted-MLWR), which drastically boosts the Core-SVP complexity of the underlying lattice. To handle and precisely control the resulting Decryption Failure Rate (DFR), we leverage a hierarchical error-correction framework—a double-layer BCH-plus-Repetition Code construction across all parameter sets, including our Level 1 instance, WEAVER-640—to safely absorb the noise.

As a result, WEAVER achieves a dual victory: it delivers a significantly higher estimated quantum lattice-security level (e.g., 137 bits for our Level 1 instance, WEAVER-640) to restore a robust security margin, while simultaneously maintaining highly competitive computational performance and achieving highly optimized transmission sizes.

1.1 Main Features and Advantages

The distinct advantages of WEAVER stem from three primary theoretical and engineering innovations.

Hierarchical Encoding and Error Correction. Aggressively reducing the ciphertext compression parameters of a module-lattice scheme substantially increases the decryption error rate, making conventional Kyber-style reconciliation insufficient for achieving a negligible failure probability. A natural remedy is to introduce error-correcting codes (ECCs). However, this encounters a structural capacity limitation of ring-based schemes: a single ciphertext polynomial provides only a limited number of reliable embedding positions, namely those in the

most-significant-bit (MSB) layer used by Kyber, where each message bit is carried near the half-modulus point $q/2$. Because ECC encoding expands the payload beyond the original session key, a single-layer embedding strategy cannot always fit the entire protected codeword into the available positions.

We address this tension by adding a second embedding layer at a lower magnitude, centered around the quarter-modulus point $q/4$, so that both levels of each coefficient carry information. The highly reliable $q/2$ layer carries most of the encoded bits, while the smaller portion of redundancy bits that overflow it is offloaded to the less reliable $q/4$ layer. This resolves the capacity bottleneck without increasing the ring dimension or the ciphertext size.

The key challenge then becomes the asymmetric reliability of the two layers: the inner $q/4$ layer is considerably more vulnerable to compression and decryption noise due to its smaller decoding margin. To compensate, we apply unequal error protection by assigning only a small fraction of bits to the inner layer and protecting them with a stronger concatenated code based on $\mathbb{Z}^4$ repetition and BCH correction, whereas the outer layer requires only moderate BCH protection. This design allows both layers to achieve similarly negligible post-decoding failure probabilities despite operating under substantially different noise conditions. For WEAVING-640, the double-layer strategy is fully deployed with $n = 128$ and $k = 5$, where the outer layer is protected by a shortened BCH(127, 113, 2) code, and the inner layer is protected by a concatenated BCH(31, 21, 2) plus 4-way repetition code, ensuring an overall decryption failure rate (DFR) below $2^{-166.3}$.

Prime-Modulus LWR and Randomized Lifting. To maximize compactness, WEAVING treats compression not merely as a bandwidth optimization, but as a Module-LWR-style hardness source. However, we insist on using NTT-friendly prime moduli ($q = 3329$ for WEAVING-640, and $q = 7681$ for WEAVING-1024 and WEAVING-2048) to maintain high arithmetic performance; schemes relying on power-of-two moduli (such as Saber [19]) typically suffer a $2\times$ performance penalty compared to ML-KEM due to the lack of native NTT support.

Combining prime-modulus arithmetic with aggressive public-key compression introduces a critical proof obstacle. In a standard LWR reduction, the public matrix must be uniformly random. If we compress a public key over a prime modulus and then deterministically decompress it, the resulting lifted values are restricted to a sparse set of canonical representatives and are distinctly non-uniform. This mismatch prevents a direct reduction to standard hardness assumptions. WEAVING elegantly resolves this via *randomized lifting*. Conditioned on the compressed value y , $\text{Inv}_q(y, d)$ samples uniformly from the preimage bucket $\text{Preim}_q(y, d)$. When y is obtained by compressing a uniform ring element, this bucket-uniform lifting restores a uniform marginal representative, which is the property used in the game transition, bridging the gap between prime-modulus efficiency and LWR compactness.

Embed-then-Round Ciphertexts and the Shifted-MLWR Assumption. Even with randomized lifting for the public key, generating the ciphertext presents another theoretical challenge. Because the ciphertext component contains a public message-dependent shift before rounding, the proof must handle the distribution of rounded shifted MLWR samples rather than standard unshifted rounding. To ensure proper masking, WEAVING embeds the message before rounding, computing the ciphertext component as $c_2 = \text{Compress}_q(v + \text{MsgEncode}(\mu), d_v)$.

While this embed-then-round approach guarantees correct message masking, it significantly complicates the security proof, as the standard MLWR assumption applies to rounding **without** an embedded message shift. To rigorously prove the security of this structure, we introduce the **Shifted-MLWR** assumption. We formally reduce the IND-CPA security of WEAVING to this Shifted-MLWR problem. Furthermore, we demonstrate that this shifted variant is not an ad-hoc assumption: by extending the Liu–Wang LWE-to-LWR proof framework [32], we provide a rigorous reduction from the standard MLWE problem to Shifted-MLWR. This theoretical frame-

work establishes that the statistical closeness holds before the deterministic shifted-rounding is applied, allowing us to safely leverage the bandwidth savings of aggressive ciphertext rounding while maintaining a solid foundation on standard lattice assumptions.

2 Algorithm Description

2.1 Preliminaries and Notation

Modular reductions. For an even (resp. odd) positive integer α , we define $r_0 = r \bmod^\pm \alpha$ to be the unique element r_0 in the range $-\frac{\alpha}{2} < r_0 \leq \frac{\alpha}{2}$ (resp. $-\frac{\alpha-1}{2} \leq r_0 \leq \frac{\alpha-1}{2}$) such that $r_0 = r \bmod \alpha$. For any positive integer α , we define $r_0 = r \bmod^+ \alpha$ to be the unique element r_0 in the range $0 \leq r_0 < \alpha$ such that $r_0 = r \bmod \alpha$. When the exact representation is not important, we simply write $r \bmod \alpha$.

Polynomial Rings. Let $\mathbb{Z}$ denote the ring of integers. For a positive integer q , let $\mathbb{Z}_q = \mathbb{Z}/q\mathbb{Z}$. We denote its **least nonnegative residues** as $\mathbb{Z}_q^+ := \{0, 1, \dots, q-1\}$. Let n be a power of 2. In our concrete parameter sets, we use $n = 128$, $n = 256$, and $n = 512$; the exact value for each parameter set is listed in Table 2. We define $\mathcal{R}_q = \mathbb{Z}_q[X]/(X^n + 1)$. Elements in the ring, $a \in \mathcal{R}_q$ are polynomials of degree less than n , which can be written as: $a = \sum_{i=0}^{n-1} a_i X^i$, $a_i \in \mathbb{Z}_q$, or equivalently denoted by its coefficient vector $a = (a_0, a_1, \dots, a_{n-1})$. When we want to uniquely decide its representation or emphasize its residue representatives, for example, we use $\mathcal{R}_{q^+}$ to denote the representation $a = (a_0, a_1, \dots, a_{n-1})$, $a_i \in \mathbb{Z}_q^+$.

Vectors and Matrices. We use bold lower-case letters (e.g., $\mathbf{u}, \mathbf{v}, \mathbf{s}$) for column vectors over $\mathcal{R}_q$, and bold upper-case letters (e.g., $\mathbf{A}$) for matrices over $\mathcal{R}_q$. For $\mathbf{u}, \mathbf{v} \in \mathcal{R}_q^k$, their inner product is

$$\mathbf{u}^T \mathbf{v} = \sum_{i=0}^{k-1} u_i v_i \in \mathcal{R}_q.$$

The transpose of a vector or matrix is denoted by $\mathbf{v}^T$ or $\mathbf{A}^T$. In the NTT domain, $\mathbf{u} \circ \mathbf{v}$ denotes coefficient-wise multiplication.

NTT-domain Representation. For an object over $\mathcal{R}_q$ —including a ring element, a vector, or a matrix—we use a hat to denote its Number Theoretic Transform (NTT) representation. Thus, for $a \in \mathcal{R}_q$, $\hat{a}$ denotes its NTT-domain form; for $\mathbf{s} \in \mathcal{R}_q^k$, $\hat{\mathbf{s}}$ denotes the vector obtained by applying the NTT component-wise; and for $\mathbf{A} \in \mathcal{R}_q^{k \times k}$, $\hat{\mathbf{A}}$ denotes the matrix obtained by applying the NTT to each polynomial entry. The NTT is used only as an efficient representation for polynomial arithmetic and does not change the underlying ring-domain objects or their probability distributions. In security proofs, we usually write the ring-domain objects a , $\mathbf{s}$, and $\mathbf{A}$ and suppress their NTT-domain representations unless the implementation detail is relevant.

Distributions and Sampling. We write $x \leftarrow \$ S$ for uniform sampling from a finite set S . Let CBD_η denote the centered binomial distribution with parameter η . A sample from CBD_η is computed as

$$\sum_{i=1}^{\eta} (a_i - b_i),$$

where each $a_i, b_i \in \{0, 1\}$ is sampled uniformly and independently. We write $x \leftarrow \text{CBD}_\eta$ to denote sampling from this distribution. When sampling a polynomial in $\mathcal{R}_q$, all coefficients are sampled independently from CBD_η .

Compression and Canonical Decompression. For $x \in \mathbb{R}$, $\lceil x \rceil$ denotes rounding to the nearest integer, with ties broken upward, and $\lfloor x \rfloor$ denotes the floor function. These operations are extended coefficient-wise to polynomials and vectors.

For $x \in \mathbb{Z}_q$ and a compression parameter $d < \lceil \log_2 q \rceil$, define

$$\text{Compress}_q(x, d) := \left\lceil \frac{2^d}{q} \cdot (x \bmod^+ q) \right\rceil \bmod^+ 2^d,$$

and, for $y \in \mathbb{Z}_{2^d}$,

$$\text{Decompress}_q(y, d) := \left\lfloor \frac{q}{2^d} \cdot (y \bmod^+ 2^d) \right\rfloor \bmod^+ q.$$

These maps extend coefficient-wise to polynomials and vectors.

Randomized Lifting. We apply Randomized Lifting technique [14] for public key's decompression, in order to achieve strict security reduction. Namely, for each $y \in \mathbb{Z}_{2^d}^+$, define its preimage set:

$$\text{Preim}_q(y, d) := \{x \in \mathbb{Z}_q^+ : \text{Compress}_q(x, d) = y\}.$$

The randomized lifting map $\text{Inv}_q(\cdot, d) : y \mapsto x'$ is defined by:

$$\text{Inv}_q(y, d) := x' \bmod^+ q, \quad x' \leftarrow \$ \text{Preim}_q(y, d).$$

That is, $\text{Inv}_q(y, d)$ outputs a uniformly random element from the set of all residues in $\mathbb{Z}_q^+$ that compress to y . For polynomials and vectors, $\text{Inv}_q(\cdot, d)$ is applied coefficient-wise and independently across coefficients unless stated otherwise.

An alternative (equivalent) approach to defining $\text{Inv}_q(\cdot, d)$ is to first define “preimage error set” as:

$$\text{Preim}_q^\pm(y, d) = \{x - \text{Decompress}_q(y, d) \bmod^\pm q, \ x \in \mathbb{Z}_q^+ : \text{Compress}_q(x, d) = y\}.$$

then define $\text{Inv}_q(\cdot, d)$ as:

$$\text{Inv}_q(y, d) := \text{Decompress}_q(y, d) + e, \quad e \leftarrow \$ \text{Preim}_q^\pm(y, d)$$

In the implementation, this per-coefficient uniform sampling is made deterministic via a pseudorandom byte stream. Let $\rho = \text{PRF}(\text{seed})$ and let c be a coefficient counter. The procedure $\text{SampleIndex}(\rho, c, m)$ returns an index uniformly distributed in $\{0, \dots, m-1\}$, computed as follows: it reads successive $\lceil \log_2 m \rceil$ -bit blocks from the substream ρ_c (the bytes of ρ associated with counter c via domain separation) and outputs the first block whose value is less than m (rejection sampling). This is the same rejection-sampling principle used to map uniform bytes to integers modulo q elsewhere in WEAVING. Since ρ is a deterministic function of seed , the map Inv_q is itself deterministic in seed , as required by the FO transform.

Now, instead of using the canonical decompression for pk recover in $\text{WEAVING.CPAPKE.Enc}$, which was $\mathbf{t}' = \text{Decompress}_q(\text{Compress}_q(\mathbf{t}, d), d)$, we apply the randomized lift:

$$\mathbf{t}_{\text{lift}} \leftarrow \text{Inv}_q(\text{Compress}_q(\mathbf{t}, d), d).$$

The Property Achieved by Randomized Lifting. By incorporating $\text{Inv}_q(\cdot, d)$, the public key after decompression in the encryption function recovers to be uniformly distributed over $\mathbb{Z}_q^+$. More precisely, if

$$x \leftarrow \$ \mathbb{Z}_q^+, \quad y = \text{Compress}_q(x, d), \quad \tilde{x} \leftarrow \text{Inv}_q(y, d),$$

then $\tilde{x}$ is uniformly distributed over $\mathbb{Z}_q^+$. Indeed, for every $a \in \mathbb{Z}_q^+$, letting $y_a = \text{Compress}_q(a, d)$, we have

$$\Pr[\tilde{x} = a] = \Pr[y = y_a] \cdot \Pr[\tilde{x} = a \mid y = y_a] = \frac{|\text{Preim}_q(y_a, d)|}{q} \cdot \frac{1}{|\text{Preim}_q(y_a, d)|} = \frac{1}{q}.$$

Hence it follows an exact uniform distribution $\tilde{x} \leftarrow \$ \mathbb{Z}_q^+$. The same statements extend coefficient-wise to polynomials in $\mathcal{R}_q$ and to vectors over $\mathcal{R}_q$.

Repetition Code ($\mathbb{Z}^4$). A 4-way repetition code with soft-decision decoding was introduced by NewHope [6], a KEM scheme which has entered the 2nd round of NIST competition. Specifically, each message bit is embedded into four distant polynomial coefficients at indices $i, i + stp, i + 2 \cdot stp$, and $i + 3 \cdot stp$. During decryption, instead of making hard decisions on individual coefficients, the receiver aggregates the absolute distances (L_1 norm) across all four dimensions before thresholding. This multi-dimensional soft-decision approach provides a substantial coding gain against the heavy rounding noise, which is then combined with the outer BCH decoding to achieve a cryptographically negligible decryption failure probability under the target parameter sets.

BCH code. A binary Bose–Chaudhuri–Hocquenghem code [34, 31] is denoted by $\text{BCH}(n_c, k_c, t_c)$, where n_c is the block length, k_c is the message length, and t_c is the error-correcting capability. For each layer $c \in \{1, 2\}$, we denote by $\text{ECCEncode}_c : \{0, 1\}^{k_c} \rightarrow \{0, 1\}^{n_c}$ and $\text{ECCDecode}_c : \{0, 1\}^{n_c} \rightarrow \{0, 1\}^{k_c}$ the deterministic encoder and decoder associated with that BCH code. In the algorithms, these maps are applied to the layer payload bits and the resulting codewords are used in Algorithm 4 and Algorithm 6. In our parameter sets, BCH coding is used to reduce the effective decryption error induced by aggressive compression while preserving compact ciphertext size.

Concatenated Code. In coding theory, codes with different capabilities can be combined in a nested manner. Concatenated code means the message is encoded using two (or more) coding schemes together to obtain efficiency that cannot be achieved by either single one of them. Precisely, during the encoding, the message is first encoded with the first scheme, which results in a codeword of the first scheme; then this intermediate codeword is further encoded with the second scheme, treated as the message payload of the second scheme. In such case, we call the codeword of the second scheme as the codeword of the concatenated code of both schemes. Actually, in the latest version of LAC [33] proposed in their paper, they also applied the idea of using concatenated code of BCH and $\mathbb{Z}^2$ repetition code.

2.2 Data Encoding and Serialization

To transmit cryptographic objects (such as polynomials and vectors) over a network, they must be serialized into byte arrays. Furthermore, since our error correction codes operate on bits, we require explicit conversions between byte arrays and bit strings. We use $\mathcal{B}$ to denote the byte alphabet $\{0, \dots, 255\}$, and $\mathcal{B}^\ell$ to denote the set of byte strings of length ℓ .

BytesToBits and BitsToBytes. For a byte array $B \in \mathcal{B}^\ell$, the function $\text{BytesToBits}(B)$ outputs an array of 8ℓ bits. The conversion is done in little-endian order per byte: the i -th byte $B[i]$ is decomposed into 8 bits $b_{8i}, b_{8i+1}, \dots, b_{8i+7}$, where b_{8i} is the least significant bit of $B[i]$. Conversely, $\text{BitsToBytes}(\mathbf{b})$ takes an array of 8ℓ bits and packs every 8 bits into a byte in the same little-endian manner to output $B \in \mathcal{B}^\ell$. In our algorithms, when we slice a byte array (e.g., $\mu[0 : \ell_1/8 - 1]$), it implicitly assumes the array can be treated as a bit string via these functions.

Encode_d. Let $f \in R_{2^d}$ be a polynomial whose coefficients $f_0, f_1, \dots, f_{n-1}$ are integers in $\{0, \dots, 2^d - 1\}$. The function $\text{Encode}_d(f)$ serializes f into an array of $n \cdot d/8$ bytes. The encoding process concatenates the d -bit representations of the coefficients in little-endian order. Specifically, we consider the sequence of $n \cdot d$ bits formed by taking the least significant bit to the most significant bit of f_0 , followed by the bits of f_1 , and so on up to f_{n-1} . This bit sequence is then partitioned into blocks of 8 bits, where each block forms a byte.

Mathematically, let the output byte array be $B \in \mathcal{B}^{nd/8}$. The m -th bit of the byte array (where $m = 8 \cdot i + j$ for $i \in \{0, \dots, nd/8 - 1\}$ and $j \in \{0, \dots, 7\}$) corresponds to the y -th bit of the coefficient f_x , where $m = x \cdot d + y$ for $y \in \{0, \dots, d - 1\}$ and $x \in \{0, \dots, n - 1\}$. The function Decode_d is defined as the exact inverse of Encode_d , and both are parameterized by the same bit-width d .

When applied to a vector of polynomials $\mathbf{v} = (v_0, \dots, v_{k-1}) \in R^k$, $\text{Encode}_d(\mathbf{v})$ simply concatenates the byte arrays obtained by applying Encode_d to each polynomial sequentially:

$$\text{Encode}_d(\mathbf{v}) = \text{Encode}_d(v_0) \parallel \text{Encode}_d(v_1) \parallel \dots \parallel \text{Encode}_d(v_{k-1})$$

resulting in an array of $n \cdot d \cdot k/8$ bytes.

Decode_d. The function $\text{Decode}_d(B)$ is the exact inverse of Encode_d . It takes an array of $n \cdot d/8$ bytes and reconstructs a polynomial $f \in R$ with coefficients in $\{0, \dots, 2^d - 1\}$. It sequentially reads d bits from the byte array B (in little-endian order) to reconstruct the integer value of each coefficient $f_0, f_1, \dots, f_{n-1}$. Similarly, for a byte array representing a vector $\mathbf{v} \in R^k$, it consumes chunks of $n \cdot d/8$ bytes to reconstruct each of the k polynomials sequentially.

2.3 Algorithm Specification

2.3.1 Specification of IND-CPA-secure PKE

Description of the PKE Algorithms. The IND-CPA-secure Public Key Encryption (PKE) scheme follows the Module-LWR framework but introduces a randomized lifting mechanism (WEAVER-Inv) and a highly novel message encoding strategy to achieve exceptional noise tolerance.

Key Generation and Encryption (Algorithms 1 and 2): Unlike standard MLWE schemes that sample explicit error vectors, WEAVER-Inv adopts a pure rounding-based (MLWR-type) approach. The public matrix $\hat{\mathbf{A}}$ is generated in the NTT domain from a public matrix-expansion function XOF_{pub} , which outputs a deterministic byte string derived from the public seed seed_A and the matrix indices. That byte string is then parsed into $\hat{\mathbf{A}}$ by a fixed rejection-sampling procedure. The secret vector $\mathbf{s}$ and the ephemeral secret $\mathbf{r}$ are sampled from a centered binomial distribution (CBD). No explicit error vectors $(\mathbf{e}, \mathbf{e}', \mathbf{e}'')$ are added. Instead, cryptographic hardness is derived entirely from the aggressive truncation of the lower bits using the Compress_q function.

Crucially, during encryption, the sender does not reconstruct the public key using a deterministic decompression. Instead, it applies the randomized lifting map Inv_q to sample a uniformly random preimage $\hat{\mathbf{t}}$ from the compression bucket. To ensure the encryption process is strictly deterministic for a given random coin seed_1 —a strict prerequisite for applying the Fujisaki-Okamoto (FO) transform—the randomness used for this lifting is explicitly derived from seed_1 via a PRF. This randomized lifting is essential for the joint-distribution property relied upon in the security proof.

Message Encoding (Algorithm 4): Unlike standard schemes that encode 1 bit per polynomial coefficient, our MsgEncode algorithm employs a *Multi-Level Coding* strategy combined with explicit error correction. The input message μ is split into two parts: $\tilde{\mu}$ (destined for the higher bits) and $\check{\mu}$ (destined for the lower bits). Both parts are independently encoded using BCH codes to produce codewords $\tilde{\mu}$ and $\check{\mu}$, respectively.

- *Higher Bits Embedding:* The bits of $\tilde{\mu}$ are embedded into the most significant part of the coefficients by multiplying them by $\frac{q-1}{2}$.
- *Lower Bits Embedding (Repetition Code):* The bits of $\check{\mu}$ are embedded into the lower significant part by multiplying by $\lfloor q/4 \rfloor$. Crucially, to ensure robustness against high noise,

each bit of μ is spread across four distant coefficients (indices $i, i+stp, i+2*stp, i+3*stp$), where stp denote the repetition stride listed in Table 3. This 4-way repetition effectively maximizes the Euclidean distance between valid codewords in the multi-dimensional space.

Message Decoding and Deterministic Decryption (Algorithms 5 and 6): For decryption, the receiver simply uses the standard deterministic canonical decompression Decompress_q to lift the ciphertext components. This asymmetric design — randomized lifting during encryption and deterministic decompression during decryption — preserves the exact uniformity required for the security proof while maintaining a naturally deterministic decryption algorithm, seamlessly supporting the standard FO transform.

To reliably recover μ from $w' \approx w + \text{noise}$, MsgDecode utilizes a *Successive Interference Cancellation (SIC)* approach:

1. *Extracting Lower Bits:* The algorithm first isolates the lower bits by evaluating the coefficients modulo $\frac{q-1}{2}$. This operation nullifies the interference from the higher bits. It then aggregates the absolute distance (L_1 norm) from the expected lower-layer center $\lfloor q/4 \rfloor$ across the four repetition positions. A soft-decision rule is applied to recover the noisy lower-layer bit string: if the accumulated error is large ($\geq \frac{q-1}{2}$), the bit is decoded as 0; otherwise, 1.
2. *First-stage Error Correction:* The extracted lower bits μ' are passed through the BCH decoder to correct any remaining errors, yielding the exact original lower message $\hat{\mu}$.
3. *Interference Cancellation:* The corrected lower bits are re-encoded to reconstruct the exact lower-layer contribution, which is then explicitly subtracted from the noisy polynomial w' to obtain $\bar{w}$.
4. *Extracting Higher Bits:* With the lower-bit interference completely removed, the higher bits can now be safely extracted using standard rounding $\lceil (2/q)\bar{w}_i \rceil$.
5. *Second-stage Error Correction:* Finally, the extracted higher bits are passed through the second BCH decoder to recover $\bar{\mu}$.
6. *Concatenate into Original Message:* Putting the bits of $\bar{\mu}$ and $\hat{\mu}$ together recovers the original encoded message μ .

Algorithm 1 WEAVING.CPAPKE.KeyGen(1^λ): Key Pair Generation

Output: Secret key $sk = \hat{s} \in \mathcal{R}_q^k \rightarrow \mathcal{B}^{\lceil \log_2 q \rceil \cdot k \cdot n/8}$

Output: Public key $pk = (t' \in \mathcal{R}_{2^{d_t}}^k, seed_A) \rightarrow \mathcal{B}^{d_t \cdot k \cdot n/8 + \text{len}_{seed}}$

```
1:  $seed_0 \leftarrow \mathcal{B}^{\text{len}_{seed}}$ 
2:  $(seed_A, seed_{sk}) = \text{XOF}(seed_0)$   $\triangleright seed_A, seed_{sk} \in \mathcal{B}^{\text{len}_{seed}}$ 
3: for  $i$  from 0 to  $k - 1$  do
4:   for  $j$  from 0 to  $k - 1$  do
5:      $\hat{A}[i][j] \leftarrow \text{XOF}_{\text{pub}}(seed_A, i, j)$   $\triangleright$  Generate matrix  $\hat{A} \in \mathcal{R}_q^{k \times k}$  in NTT domain
6:   end for
7: end for
8:  $ctr = 0$ 
9: for  $i$  from 0 to  $k - 1$  do
10:   $s[i] \leftarrow \text{CBD}_{\eta_1}(\text{PRF}(seed_{sk}, ctr))$   $\triangleright$  Sample  $s \in \mathcal{R}_q^k$ 
11:   $ctr = ctr + 1$ 
12: end for
13:  $\hat{s} = \text{NTT}(s)$ 
14:  $t = \text{NTT}^{-1}(\hat{A} \circ \hat{s})$   $\triangleright$  Pure MLWR: no explicit error  $e$  is added
15:  $sk = \text{Encode}_{\lceil \log_2 q \rceil}(\hat{s})$ 
16:  $pk = (\text{Encode}_{d_t}(\text{Compress}_q(t, d_t)), seed_A)$ 
17: return  $(pk, sk)$ 
```

Algorithm 2 WEAVING.CPAPKE.Enc($pk, \mu; seed_1$): Encryption

Input: Public key $pk \in \mathcal{B}^{d_t \cdot k \cdot n/8 + \text{len}_{seed}}$

Input: Message $\mu \in \mathcal{B}^{\text{len}_m}$

Input: Random coins $seed_1 \in \mathcal{B}^{\text{len}_{seed}}$ $\triangleright$ Seed for PRF; the byte stream is partitioned by counter

Output: Ciphertext $ct = (c_1, c_2) \in ((\mathbb{Z}_{2^{d_u}}^n)^k, \mathbb{Z}_{2^{d_v}}^n) \rightarrow \mathcal{B}^{(d_u \cdot k + d_v) \cdot n/8}$

```
1:  $t' \leftarrow \text{Decode}_{d_t}(pk[0 : d_t \cdot k \cdot n/8])$   $\triangleright$  Decode the compressed public-key vector
2:  $seed_A \leftarrow pk[d_t \cdot k \cdot n/8 : d_t \cdot k \cdot n/8 + \text{len}_{seed}]$   $\triangleright$  Extract the matrix seed
3:  $ctr = 0$ 
4:  $t_{\text{lift}} = \text{Inv}_q(t', d_t; \text{PRF}(seed_1, ctr))$   $\triangleright$  Derandomized lifting using  $seed_1$ 
5:  $ctr = ctr + 1$ 
6:  $\hat{t} = \text{NTT}(t_{\text{lift}})$ 
7: for  $i$  from 0 to  $k - 1$  do
8:   for  $j$  from 0 to  $k - 1$  do
9:      $\hat{A}^T[i][j] \leftarrow \text{XOF}_{\text{pub}}(seed_A, j, i)$   $\triangleright$  Generate the transpose of matrix  $\hat{A}$ 
10:   end for
11: end for
12: for  $i$  from 0 to  $k - 1$  do
13:   $r[i] \leftarrow \text{CBD}_{\eta_2}(\text{PRF}(seed_1, ctr))$   $\triangleright$  Sample  $r \in \mathcal{R}_q^k$ 
14:   $ctr = ctr + 1$ 
15: end for
16:  $\hat{r} = \text{NTT}(r)$ 
17:  $w = \text{MsgEncode}(\mu)$   $\triangleright w \in \mathbb{Z}_q^n$ 
18:  $u = \text{NTT}^{-1}(\hat{A}^T \circ \hat{r})$   $\triangleright$  No explicit error  $e'$ 
19:  $v_0 = \text{NTT}^{-1}(\hat{t}^T \circ \hat{r})$   $\triangleright$  No explicit error  $e''$ 
20:  $c_1 = \text{Encode}_{d_u}(\text{Compress}_q(u, d_u))$ 
21:  $c_2 = \text{Encode}_{d_v}(\text{Compress}_q(v_0 + w, d_v))$   $\triangleright$  Add message in  $\mathcal{R}_q$  before compression
22: return  $ct = (c_1 \parallel c_2)$ 
```

Algorithm 3 $\text{Inv}_q(t', d_t; \text{seed})$: Randomized Lifting

Input: Compressed public key $t' \in \mathcal{R}_{2^{d_t}}^k$ **Input:** Decompress parameter d_t **Input:** Random coins $\text{seed} \in \mathcal{B}^{\text{len}_{\text{seed}}}$ **Output:** Re-randomized decompressed public key $t_{\text{lift}} \in \mathcal{R}_q^k$

```
1:  $\rho = \text{PRF}(\text{seed})$  ▷ Deterministic pseudorandom byte stream
2: for  $i$  from 0 to  $k - 1$  do
3:   for  $j$  from 0 to  $n - 1$  do
4:      $B = \text{Preim}_q^\pm(t'^{(i)}[j], d_t)$  ▷ Error bucket for this coefficient
5:      $\iota = \text{SampleIndex}(\rho, i \cdot n + j, |B|)$  ▷ Index in  $\{0, \dots, |B| - 1\}$  from  $\rho$ 
6:      $e = B[\iota]$  ▷ Deterministic selection from the bucket
7:      $t_{\text{lift}}^{(i)}[j] = \text{Decompress}_q(t'^{(i)}[j], d_t) + e$ 
8:   end for
9: end for
10: return  $t_{\text{lift}}$ 
```

Algorithm 4 $\text{MsgEncode}(\mu)$: Message Encoding

Input: Message bits $\mu \in \mathcal{B}^{\text{len}_m}$ **Output:** Encoded element $w \in \mathbb{Z}_q^n$

```
1:  $\bar{\mu} := \mu[0 : \ell_1/8 - 1]$  ▷ Message bits to be encoded to higher layer
2:  $\dot{\mu} := \mu[\ell_1/8 : \text{len}_m - 1]$  ▷ Message bits to be encoded to lower layer
3:  $\tilde{\mu} = \text{ECCEncode}_1(\bar{\mu})$  ▷ Higher-layer BCH encoding
4:  $\ddot{\mu} = \text{ECCEncode}_2(\dot{\mu})$  ▷ Lower-layer BCH encoding
5:  $w = 0 \in \mathbb{Z}_q^n$ 
6: for  $i$  from 0 to  $n - 1$  do ▷ Encode to higher layer in compressed domain
7:    $w_i = w_i + \frac{q-1}{2} \cdot \tilde{\mu}_i$ 
8: end for
9: for  $i$  from 0 to  $\ell_3 - 1$  do ▷ Encode to lower layer in compressed domain
10:   $w_i = w_i + \lfloor q/4 \rfloor \cdot \ddot{\mu}_i$ 
11:   $w_{i+stp} = w_{i+stp} + \lfloor q/4 \rfloor \cdot \ddot{\mu}_i$ 
12:   $w_{i+2 \cdot stp} = w_{i+2 \cdot stp} + \lfloor q/4 \rfloor \cdot \ddot{\mu}_i$ 
13:   $w_{i+3 \cdot stp} = w_{i+3 \cdot stp} + \lfloor q/4 \rfloor \cdot \ddot{\mu}_i$ 
14: end for
15: return  $w$ 
```

Algorithm 5 $\text{WEAVER.CPAPKE.Dec}(sk, ct)$: Decryption

Input: Secret key $sk \in \mathcal{B}^{\lceil \log_2 q \rceil \cdot k \cdot n/8}$ **Input:** Ciphertext $ct \in \mathcal{B}^{(d_u \cdot k + d_v) \cdot n/8}$ **Output:** Message $\mu \in \mathcal{B}^{\text{len}_m}$

```
1:  $c_1 \leftarrow ct[0 : d_u \cdot k \cdot n/8]$  ▷ Extract the encoded first ciphertext component
2:  $c_2 \leftarrow ct[d_u \cdot k \cdot n/8 : (d_u \cdot k + d_v) \cdot n/8]$  ▷ Extract the encoded second ciphertext component
3:  $u = \text{Decompress}_q(\text{Decode}_{d_u}(c_1), d_u)$  ▷ Deterministic canonical decompression
4:  $v = \text{Decompress}_q(\text{Decode}_{d_v}(c_2), d_v)$ 
5:  $\hat{s} = \text{Decode}_{\lceil \log_2 q \rceil}(sk)$ 
6:  $w' = v - \text{NTT}^{-1}(\hat{s}^T \circ \text{NTT}(u)) \bmod^+ q$ 
7:  $\mu = \text{MsgDecode}(w')$ 
8: return  $\mu$ 
```

Algorithm 6 MsgDecode(w): Message Decoding

Input: Decrypted ring element $w \in \mathcal{R}_q$

Output: Decoded Message bits $\mu \in \mathcal{B}^{\text{len}_m}$

```

1: for  $i$  from 0 to  $\ell_3 - 1$  do                                     ▷ Decode from lower layer
2:    $ee = \left\lfloor w_i \bmod^+ \frac{q-1}{2} - \lfloor q/4 \rfloor \right\rfloor$ 
3:    $ee = ee + \left\lfloor w_{i+stp} \bmod^+ \frac{q-1}{2} - \lfloor q/4 \rfloor \right\rfloor$ 
4:    $ee = ee + \left\lfloor w_{i+2 \cdot stp} \bmod^+ \frac{q-1}{2} - \lfloor q/4 \rfloor \right\rfloor$ 
5:    $ee = ee + \left\lfloor w_{i+3 \cdot stp} \bmod^+ \frac{q-1}{2} - \lfloor q/4 \rfloor \right\rfloor$ 
6:    $\mu'_i = (ee \geq \frac{q-1}{2}) ? 0 : 1$ 
7: end for
8:  $\dot{\mu} = \text{ECCDecode}_2(\mu')$                                      ▷ Lower-layer BCH decoding
9:  $\ddot{\mu} = \text{ECCEncode}_2(\dot{\mu})$ 
10:  $\bar{w} = w$ 
11: for  $i$  from 0 to  $\ell_3 - 1$  do                                     ▷ Remove lower-layer contribution
12:    $\bar{w}_i = \bar{w}_i - \lfloor q/4 \rfloor \cdot \ddot{\mu}_i$ 
13:    $\bar{w}_{i+stp} = \bar{w}_{i+stp} - \lfloor q/4 \rfloor \cdot \ddot{\mu}_i$ 
14:    $\bar{w}_{i+2 \cdot stp} = \bar{w}_{i+2 \cdot stp} - \lfloor q/4 \rfloor \cdot \ddot{\mu}_i$ 
15:    $\bar{w}_{i+3 \cdot stp} = \bar{w}_{i+3 \cdot stp} - \lfloor q/4 \rfloor \cdot \ddot{\mu}_i$ 
16: end for
17: for  $i$  from 0 to  $n - 1$  do                                     ▷ Decode from higher layer
18:    $\mu''_i = \left\lceil \frac{2}{q} \cdot \bar{w}_i \right\rceil \bmod 2$ 
19: end for
20:  $\bar{\mu} = \text{ECCDecode}_1(\mu'')$                                      ▷ Higher-layer BCH decoding
21: return  $\mu := (\bar{\mu} \parallel \dot{\mu})$ 

```

2.3.2 Specification of IND-CCA-secure KEM

To achieve IND-CCA security, we apply a variant of the Fujisaki-Okamoto (FO) transform to our underlying IND-CPA-secure PKE scheme. This transform ensures that any modification to the ciphertext will result in a pseudorandom shared secret, providing implicit rejection of invalid ciphertexts. We utilize three cryptographic hash functions modeled as random oracles: H , G , and J . Their security strengths and output lengths, together with those of the seed-derivation extendable-output function XOF , the public matrix-expansion function XOF_{pub} , and the pseudorandom function PRF , are specified per security level in Section 2.4.

KEM Algorithms. The KEM consists of three algorithms: $\text{WEAVER.CCAKEM.KeyGen}$, $\text{WEAVER.CCAKEM.Encaps}$, and $\text{WEAVER.CCAKEM.Decaps}$, which are detailed in Algorithms 7, 8, and 9, respectively. The byte length of both PKE's encrypted message and KEM's shared secret equals $\text{len}_{\text{ss}} = |\mu|/8$. The hash-output length len_H and the matrix-seed length len_{seed} are set per security level; their required security strengths are given in Section 2.4.

Algorithm 7 $\text{WEAVER.CCAKEM.KeyGen}(1^\lambda)$: KEM Key Generation

Output: KEM Secret key $sk_{KEM} \in \mathcal{B}^{12 \cdot k \cdot n/8 + d_t \cdot k \cdot n/8 + \text{len}_{\text{seed}} + \text{len}_H + \text{len}_{\text{ss}}}$

Output: KEM Public key $pk \in \mathcal{B}^{d_t \cdot k \cdot n/8 + \text{len}_{\text{seed}}}$

- 1: $z \leftarrow \$ \mathcal{B}^{\text{len}_{\text{ss}}}$ ▷ Sample implicit rejection value
 - 2: $(pk, sk_{PKE}) = \text{WEAVER.CPAPKE.KeyGen}(1^\lambda)$
 - 3: $sk_{KEM} = (sk_{PKE} \parallel pk \parallel H(pk) \parallel z)$
 - 4: **return** (pk, sk_{KEM})
-

Algorithm 8 $\text{WEAVER.CCAKEM.Encaps}(pk)$: KEM Encapsulation

Input: KEM Public key $pk \in \mathcal{B}^{d_t \cdot k \cdot n/8 + \text{len}_{\text{seed}}}$

Output: Ciphertext $ct \in \mathcal{B}^{(d_u \cdot k + d_v)n/8}$

Output: Shared secret $K \in \mathcal{B}^{\text{len}_{\text{ss}}}$

- 1: $m \leftarrow \$ \mathcal{B}^{\text{len}_{\text{ss}}}$ ▷ Sample random message
 - 2: $(K, r) = G(m \parallel H(pk))$ ▷ Derive shared secret and coins
 - 3: $ct = \text{WEAVER.CPAPKE.Enc}(pk, m; r)$ ▷ Encrypt using derived coins
 - 4: **return** (ct, K)
-

Algorithm 9 $\text{WEAVER.CCAKEM.Decaps}(sk_{KEM}, ct)$: KEM Decapsulation

Input: KEM Secret key $sk_{KEM} = (sk_{PKE} \parallel pk \parallel h_{pk} \parallel z)$

Input: Ciphertext $ct \in \mathcal{B}^{(d_u \cdot k + d_v)n/8}$

Output: Shared secret $K \in \mathcal{B}^{\text{len}_{\text{ss}}}$

- 1: $m' = \text{WEAVER.CPAPKE.Dec}(sk_{PKE}, ct)$ ▷ Decrypt the ciphertext
 - 2: $(K, r') = G(m' \parallel h_{pk})$ ▷ Re-derive shared secret and coins
 - 3: $ct' = \text{WEAVER.CPAPKE.Enc}(pk, m'; r')$ ▷ Re-encrypt to verify ciphertext
 - 4: **if** $ct \neq ct'$ **then**
 - 5: $K = J(z, ct)$ ▷ Invalid ciphertext: implicit rejection and overwrite output secret
 - 6: **end if**
 - 7: **return** K
-

Explanation of the FO Transform. The KEM Key Generation algorithm extends the PKE secret key by appending the public key pk , its hash $H(pk)$, and a uniformly random value

$z \in \mathcal{B}^{\text{len}_{\text{ss}}}$. The value z is crucial for the implicit rejection mechanism. During Encapsulation, a random message $m \in \mathcal{B}^{\text{len}_{\text{ss}}}$ is chosen and hashed. The function G acts as a random oracle to derive both a pre-master secret $\hat{K}$ and the deterministic randomness r required for the PKE encryption. The output of G provides both the candidate shared secret K and the deterministic encryption coins r . The ciphertext is subsequently verified during decapsulation by re-encryption. Thus, valid ciphertexts are accepted only when they are consistent with the derived coins, while invalid ciphertexts are mapped to the ciphertext-dependent fallback value $J(z, ct)$. In Decapsulation, the receiver decrypts the ciphertext to obtain m' and re-runs the encryption process using the re-derived randomness r' . If the re-computed ciphertext ct' matches the received ct , the decapsulation succeeds, and the valid shared secret is returned. If the check fails (indicating a potentially malicious or malformed ciphertext), the KEM does not abort but instead returns a pseudorandom key derived from the secret value z and the hash of the ciphertext. This implicit rejection prevents chosen-ciphertext attacks (CCA) from leaking information about the secret key.

2.4 Symmetric Primitives and Quantum Security Levels

WEAVER uses six symmetric primitives. For implementability, this subsection fixes their input domains, output domains, and the byte lengths consumed or produced by the algorithms in Section 4. The primitives are grouped into three roles:

- **Public expansion** (XOF_{pub}). Deterministically expands the public seed seed_A and matrix indices into a byte string that is parsed into the public matrix $\hat{\mathbf{A}}$.
- **Secret derivation and sampling** (XOF, PRF). Derive the secret seed, generate secret material, and feed the randomized lifting procedure used during encryption.
- **FO-transform hashes** ($\mathbf{H}, \mathbf{G}, \mathbf{J}$). Bind the public key and derive the encapsulation secret, ephemeral coins, and implicit-rejection fallback key.

Normative interfaces. For a fixed security level, let len_{seed} , $\text{len}_{\mathbf{H}}$, and len_{ss} be the byte lengths listed in Table 1. The symmetric primitives are defined by the following interfaces:

$$\begin{aligned} \text{XOF} &: \mathcal{B}^{\text{len}_{\text{seed}}} \rightarrow \mathcal{B}^{2\text{len}_{\text{seed}}}, \\ \text{XOF}_{\text{pub}} &: \mathcal{B}^{\text{len}_{\text{seed}}} \times \{0, \dots, k-1\}^2 \rightarrow \{0, 1\}^*, \\ \text{PRF} &: \mathcal{B}^{\text{len}_{\text{seed}}} \rightarrow \{0, 1\}^*, \\ \mathbf{H} &: \{0, 1\}^* \rightarrow \mathcal{B}^{\text{len}_{\mathbf{H}}}, \\ \mathbf{G} &: \mathcal{B}^{\text{len}_{\text{ss}} + \text{len}_{\mathbf{H}}} \rightarrow \mathcal{B}^{\text{len}_{\text{ss}} + \text{len}_{\text{seed}}}, \\ \mathbf{J} &: \mathcal{B}^{\text{len}_{\text{ss}}} \times \{0, 1\}^* \rightarrow \mathcal{B}^{\text{len}_{\text{ss}}}. \end{aligned}$$

Here $\text{XOF}(\text{seed}_0)$ is parsed as $(\text{seed}_A, \text{seed}_{sk})$, where both outputs have length len_{seed} bytes. The notation $\text{PRF}(\text{seed})$ denotes a deterministic pseudorandom byte stream keyed by seed ; the derived notation $\text{PRF}(\text{seed}, \text{ctr})$ refers to the ctr -th domain-separated substream or block of that stream, as required by the consuming primitive. Whenever a consuming primitive requires a fixed number of bytes, it takes exactly the needed prefix from the corresponding stream. In particular, CBD_{η} and $\text{Inv}_{\mathbf{q}}$ consume the prefix length they require, while $\mathbf{G}$, $\mathbf{H}$, and $\mathbf{J}$ have fixed output lengths.

Public matrix expansion. For each pair (i, j) , let $B_{i,j} = \text{XOF}_{\text{pub}}(\text{seed}_A, i, j)$. The function XOF_{pub} shall output a deterministic byte string and shall not be interpreted directly as a ring element. The corresponding matrix entry $\hat{\mathbf{A}}[i][j]$ shall be computed by the rejection-sampling

parser $\text{ParseUniform}_q(B_{i,j})$, defined as follows. Let $w = \lceil \log_2 q \rceil$. Read $B_{i,j}$ as a sequence of non-overlapping w -bit little-endian candidates. Each candidate $c \in \{0, \dots, 2^w - 1\}$ with $c < q$ is accepted as the next coefficient in $\mathbb{Z}_q^+$; each candidate with $c \geq q$ is discarded. Continue until n coefficients have been accepted. The accepted coefficients are assembled in order into the polynomial $\hat{\mathbf{A}}[i][j] \in \mathcal{R}_q$. If the byte string is exhausted before n coefficients are obtained, the caller shall continue reading from the same XOF instance with the same domain-separation inputs until enough bytes have been produced.

Security requirements. XOF_{pub} is a public, deterministic expansion function. Its output is a byte string, and the corresponding matrix coefficients are obtained by the fixed rejection-sampling parser defined above. The function does not need pre-image resistance or collision resistance. The functions XOF and PRF are secret-keyed derivation/expansion primitives and must provide pseudorandomness for their outputs. Among the FO-transform hashes, H additionally requires collision resistance because it binds the public key through $H(pk)$; G and J are required to be output-pseudorandom at the target security level, with J also acting as the ciphertext-dependent fallback for implicit rejection.

Let (λ_C, λ_Q) be the targeted classical and quantum strengths. Under the generic pre-image and collision-search bounds, the underlying primitive parameters are chosen as

$$\text{H: } b = \max(2\lambda_C, 3\lambda_Q), \quad \text{G, J, XOF, PRF: } b = \max(\lambda_C, 2\lambda_Q).$$

The public expansion function XOF_{pub} is exempt from the level-dependent bound above; any fixed-security XOF that yields the required output length may instantiate it at all security levels, provided that the induced coefficient parser is the one specified by the PKE algorithms.

For the seeds used by XOF and PRF, the seed length is set to at least $2\lambda_Q$ bits and no smaller than the required classical strength. The same seed-length rule is applied wherever these values are consumed in the algorithms, so the encodings of seed_0 , seed_A , seed_{sk} , and the encapsulation randomness remain unambiguous across implementations. We do not fix concrete instantiations; we assume idealized primitives meeting the security parameters listed in Table 1. Any concrete hash, XOF, or PRF named in the reference implementation (Section 5) is provided only as an example instantiation and is not part of the normative specification; it may be replaced by any primitive meeting the corresponding security parameters.

The KEM message space is $|\mathcal{M}| = 2^{|\mu|}$. A quantum search over $\mathcal{M}$ requires approximately $2^{|\mu|/2}$ evaluations of a verification oracle that recomputes $G(m \parallel H(pk))$, reruns WEAVER.CPAPKE.Enc , and compares the resulting ciphertext with the target ciphertext. Hence

$$C_{\text{search}} \approx 2^{|\mu|/2} \cdot C_{\text{oracle}},$$

where C_{oracle} denotes the reversible gate cost of one such verification oracle. For WEAVER-640, $|\mu| = 128$; reaching an 80-bit gate-cost estimate therefore requires $C_{\text{oracle}} \geq 2^{16}$. This is a conservative heuristic estimate: the oracle includes G, message encoding, public matrix–vector multiplication, compression, ciphertext comparison, and reversible uncomputation; the matrix–vector product alone contains $k^2n = 5^2 \cdot 128$ coefficient-level products. This message-search estimate is KEM-internal and does not refer to any external symmetric encryption using the shared secret.

2.5 Parameter Sets

To meet the diverse security requirements of different applications, we propose three parameter sets for WEAVER, as required by NGCC. We denote these instances as WEAVER-640, WEAVER-1024, and WEAVER-2048.

Table 1: Security parameters for the symmetric primitives. The parameter b denotes the security parameter of the underlying primitive, in bits. The matrix-expansion function XOF_{pub} requires only output pseudorandomness and is therefore exempt from the level-dependent bound; it may be instantiated by a fixed-security XOF such as **SHAKE128** at all levels.

Parameter	WEAVER-640	WEAVER-1024	WEAVER-2048
H security parameter b (bits)	256	512	1024
H output length 8len_{H} (bits)	256	512	1024
G, J, XOF, PRF security parameter b (bits)	256	256	512
XOF_{pub} requirement	pseudorandomness only (level-independent)		
Seed length $8\text{len}_{\text{seed}}$ (bits)	256	256	512
Shared secret length $8\text{len}_{\text{ss}} = \mu $ (bits)	128	256	512

Core Parameters. Table 2 lists the core cryptographic parameters. The parameter set WEAVER-640 uses the modulus $q = 3329$, while WEAVER-1024 and WEAVER-2048 use $q = 7681$. The ring dimension n and module rank k scale according to the targeted security level. The parameters η_1 and η_2 define the width of the centered binomial distributions (CBD) used for sampling the secret. The error vectors are sampled deterministically from rounding. To minimize bandwidth, the public key and ciphertext components are compressed to d_t, d_u , and d_v bits.

Table 2: Proposed Core Parameter Sets for WEAVER

Instance	Level	$ \mu $	n	k	q	η_1	η_2	(d_t, d_u, d_v)
WEAVER-640	1	128	128	5	3329	3	2	(9, 9, 6)
WEAVER-1024	3	256	256	4	7681	7	7	(10, 10, 8)
WEAVER-2048	5	512	512	4	7681	9	9	(11, 11, 9)

Error Correction and Decryption Failure Rate (DFR). A defining feature of WEAVER is its robust multi-level error correction mechanism, detailed in Table 3. The shared secret μ is split into two parts: $\bar{\mu}$ (encoded in the higher bits) and $\dot{\mu}$ (encoded in the lower bits via a 4-way repetition).

- For all three security levels (WEAVER-640, WEAVER-1024 and WEAVER-2048), we utilize the two-layer encoding. The lower bits are protected by a combination of BCH codes and 4-way repetition, while the higher bits are protected by another BCH code.

The overall decryption failure rate δ is bounded by the union bound over the two decoding layers, i.e., $\delta = \delta^{(1)} + \delta^{(2)}$ (with $\delta^{(2)} = 0$ for the Layer-1-only parameter set). As shown in Table 3, all parameter sets achieve a cryptographically negligible DFR (e.g., $\leq 2^{-134}$).

Effective LWE Noise and Security Amplification. When evaluating the concrete security of our scheme against lattice reduction attacks (e.g., primal and dual attacks), we consider the *effective noise* introduced by the deterministic rounding operations. In our scheme, the decompression operation introduces a rounding error e_{round} that is uniformly distributed in the interval $[-\frac{q}{2^{d+1}}, \frac{q}{2^{d+1}}]$. Therefore, the variance of the effective noise is: $\sigma_{\text{eff}}^2 \approx \frac{q^2}{12 \cdot 2^{2d}}$.

This phenomenon, closely related to the Learning With Rounding (LWR) problem, provides a significant security amplification. While schemes like ML-KEM also benefit from this, their ability to leverage rounding error is strictly bottlenecked by the need to maintain a negligible DFR without explicit error correction. In contrast, our robust BCH and 4-way repetition with soft-decision decoding mechanism allows us to employ highly aggressive compression parameters

Table 3: Error Correction Parameters and Decryption Failure Rates. The “BCH Code” lines show the triple (Codeword Length, Maximum Payload Bits, Error Tolerance), and the actual payload length can be equal or smaller than “Maximum Payload Bits”

Parameter	WEAVER-640	WEAVER-1024	WEAVER-2048
Total Message Bits $ \mu $	128	256	512
Encoding Strategy	double Layer	double Layer	double Layer
Layer 1 (Higher Bits)			
Payload Bits $\ell_1 = \bar{\mu} $	112	220	448
BCH Code (n, k, t)	(127, 113, 2)	(255, 223, 4)	(511, 448, 7)
Layer 1 DFR ($\log_2 \delta^{(1)}$)	-167.1	-233.5	-491.2
Layer 2 (Lower Bits / 4-way repetition)			
Payload Bits $\ell_2 = \dot{\mu} $	16	36	64
BCH Code (n, k, t)	(31, 21, 2)	(63, 59, 4)	(127, 78, 7)
Codeword length ℓ_3	26	60	113
step length (stp)	32	64	128
Layer 2 DFR ($\log_2 \delta^{(2)}$)	-166.3	-231.7	-488.3
Overall DFR ($\log_2 \delta$)	-166.3	-231.7	-488.3

(i.e., smaller d_t, d_u, d_v). We intentionally inject a much larger σ_{round}^2 into the LWE instances. This design rationale elegantly achieves two goals simultaneously: it drastically reduces the bandwidth requirements, while simultaneously boosting the core-SVP hardness.

Table 4 summarizes the Core-SVP security estimates (accounting for the effective noise) and the resulting sizes for the public key (pk) and ciphertext (ct). Our parameter sets comfortably exceed the security thresholds of their respective NIST categories while maintaining highly competitive bandwidth profiles.

Table 4: *Core-SVP Security* Estimates(Bit) and Data Sizes(Bytes).

Instance	Classical	Quantum	$ pk $	$ ct $
WEAVER-640	153	137	752	816
WEAVER-1024	264	237	1312	1536
WEAVER-2048	527	473	2880	3392

3 Design Rationale

3.1 Underlying Hard Problems

The security of WEAVER is rooted in the hardness of lattice problems over module lattices, specifically the Learning With Rounding (LWR) problem, its module variant (MLWR), and a structurally necessary variant we introduce as the **Shifted Module Learning With Rounding (Shifted-MLWR)** problem. Unlike traditional lattice-based schemes that rely on the addition of explicit Gaussian or binomial noise, WEAVER adopts a pure rounding-based approach to derive its cryptographic hardness. This design choice is motivated by the need for extreme bandwidth efficiency and the theoretical elegance of the randomized lifting mechanism.

Let $\mathcal{R}_q = \mathbb{Z}_q[X]/(X^n + 1)$ be the polynomial ring, and let k denote the module rank. The core security of the scheme is analyzed through the lens of the decisional MLWR and Shifted-MLWR problems.

Module Learning With Rounding (MLWR). The decisional $\text{MLWR}_{k,\ell,\eta,d}$ problem asks an adversary to distinguish between the following two distributions:

1. *Uniform distribution:* $\mathbf{A} \leftarrow \mathcal{R}_q^{\ell \times k}$ and $\mathbf{u} \leftarrow \mathcal{R}_q^\ell$ are sampled uniformly at random, and the adversary receives $(\mathbf{A}, \text{Compress}_q(\mathbf{u}, d))$;
2. *MLWR distribution:* $\mathbf{A} \leftarrow \mathcal{R}_q^{\ell \times k}$, $\mathbf{s} \leftarrow \text{CBD}_\eta^k$, and the adversary receives $(\mathbf{A}, \text{Compress}_q(\mathbf{A}\mathbf{s}, d))$.

In WEAVING, the public key is a compressed MLWR sample with $\ell = k$. The hardness of MLWR over rings with polynomial moduli is a well-studied problem. As demonstrated by [14], the hardness of such instances can be supported by the Computational Ring-LWR (R-CLWR) framework, which provides a reduction from worst-case lattice problems even when the modulus q is not exponentially large.

Decisional Shifted-MLWR and Its Equivalence. During the encryption phase of WEAVING, the second ciphertext component c_2 is generated by embedding the encoded message before rounding, yielding $c_2 = \text{Compress}_q(\mathbf{t}_{\text{lift}}^T \mathbf{r} + \text{MsgEncode}(\mu), d_v)$. To rigorously prove IND-CPA security under this embed-then-round structure, we introduce the decisional Shifted-MLWR $_{k,\ell,\eta,d,\Delta}$ problem. Here, given a fixed public offset $\Delta \in \mathcal{R}_q^\ell$, the adversary must distinguish between $(\mathbf{A}, \text{Compress}_q(\mathbf{u} + \Delta, d))$ and $(\mathbf{A}, \text{Compress}_q(\mathbf{A}\mathbf{s} + \Delta, d))$, where $\mathbf{u} \leftarrow \mathcal{R}_q^\ell$ is uniform and $\mathbf{s} \leftarrow \text{CBD}_\eta^k$.

We argue that the hardness of the Shifted-MLWR problem is concretely and asymptotically equivalent to that of the standard unshifted MLWR problem based on two key theoretical observations:

- **Geometric Invariance under Public Shift:** The public offset Δ is a known, fixed vector that is completely independent of the secret $\mathbf{s}$. In the underlying lattice $\mathcal{L} = \{\mathbf{y} \in \mathcal{R}_q^\ell \mid \mathbf{y} \equiv \mathbf{A}\mathbf{s} \pmod{q}\}$, adding Δ merely translates the entire target space by a constant vector. This translation does not alter the shortest vector structure, the volume of the fundamental domain, or any other geometric invariants of the lattice. Consequently, the difficulty of the bounded distance decoding (BDD) search space remains entirely unchanged.
- **Translation Invariance of Uniform Noise:** Statistically, because the unrounded LWE-type samples $\mathbf{A}\mathbf{s}$ possess high min-entropy, their fractional parts before rounding are extremely close to being uniformly distributed modulo the rounding width. Since the public shift Δ is a deterministic translation, applying it to a uniform distribution over a periodic interval yields another perfectly uniform distribution. Thus, the rounding error distribution in Shifted-MLWR is statistically identical in shape and variance to that of ordinary MLWR.

Furthermore, as formalized in Appendix A, we provide a theoretical reduction from standard MLWE to Shifted-MLWR by extending the Liu–Wang LWE-to-LWR proof framework [32]. This reduction establishes that statistical closeness holds before the deterministic shifted-rounding is applied. We note that while this reduction is asymptotic and serves as a strong theoretical feasibility argument rather than a tight security proof for our concrete, highly optimized parameters, it provides solid evidence that the public shift itself introduces no structural weaknesses or additional entropy leakage.

The Role of Randomized Lifting in Security. A significant challenge in compressed lattice schemes is the distribution of the public key during the encryption phase. In standard designs, the sender must use a deterministic decompression map $\text{Decompress}_q(\cdot, d)$, which produces a lifted vector that is restricted to a small subset of $\mathcal{R}_q^k$, thereby breaking the uniformity required for a tight reduction to standard MLWR.

WEAVER solves this problem via the *randomized lifting* map $\text{Inv}_q(\cdot, d)$. As established in Section 2.1, conditioned on the compressed value y , $\text{Inv}_q(y, d)$ samples uniformly from the preimage bucket $\text{Preim}_q(y, d)$. When y is obtained by compressing a uniform ring element, this bucket-uniform lifting restores a uniform marginal representative, which is the property used in the game transition. This bucket-uniformity property is the cornerstone of our security proof in Section 4.1, as it allows the simulator to replace the lifted representative by a uniform ring element in the relevant game transition without introducing statistical divergence.

Effective Noise Interpretation for Concrete Security. While the theoretical proof is anchored in the MLWR and Shifted-MLWR assumptions, for the purpose of concrete security estimation (Section 4.2), we adopt a noise-centric viewpoint. The deterministic rounding operation $\text{Compress}_q(\cdot, d)$ inherently introduces a rounding error that is approximately uniform over the quantization interval. In our analysis, this rounding error is treated as the *effective noise* of the LWE instance. The variance of this noise is given by:

$$\sigma_{\text{eff}}^2 \approx \frac{q^2}{12 \cdot 2^{2d}}.$$

By aggressively reducing the compression precision d (e.g., $d_v = 6$ for WEAVER-640), WEAVER intentionally injects a large amount of rounding noise into the system. This design philosophy, centered on *Precise Noise Engineering*, achieves two goals simultaneously: it drastically reduces the communication overhead and significantly boosts the concrete Core-SVP hardness against lattice reduction attacks.

Security Rationale Summary. In summary, the security of WEAVER is justified at three complementary levels:

- **Provable Level:** The scheme reduces directly to the standard decisional MLWR and Shifted-MLWR problems, facilitated by the randomized lifting mechanism.
- **Hardness Level:** The underlying MLWR and Shifted-MLWR instances are supported by asymptotic reductions from standard MLWE, providing strong theoretical justification for the soundness of our design.
- **Concrete Level:** The scheme is evaluated against the best known primal and dual attacks using the effective noise magnitude induced by aggressive compression, where the public shift is shown to introduce no security degradation.

This multi-layered rationale ensures that WEAVER provides robust post-quantum security while reaching the limits of bandwidth optimization.

3.2 Message Encoding Strategy and Error Correction

The most significant departure of WEAVER from standard Module-LWE based KEMs (like Kyber/ML-KEM) lies in its highly optimized message encoding mechanism. To achieve extreme compactness in ciphertext and public key sizes, WEAVER aggressively compresses the transmitted elements. This aggressive compression naturally introduces a much higher noise variance during decryption. To safely manage this noise and ensure a negligible Decryption Failure Rate (DFR), we introduce a hierarchical encoding scheme combining BCH codes with a concatenated 4-way repetition code.

Hierarchical Encoding via Information-Theoretic Channel Multiplexing. In standard lattice-based KEMs like ML-KEM, the message is encoded exclusively into the most significant bit (MSB) of the polynomial coefficients. During decapsulation, the receiver computes $v - \mathbf{s}^T \mathbf{u} \approx \text{Encode}(\mu) + \text{noise}$. To ensure strict correctness, the noise must be small enough to never flip this MSB.

We observe that the lower-order bits (e.g., the second MSB) also possess significant information-carrying capacity. From an information-theoretic perspective, this lower-bit layer can be viewed as a “noisier channel.” While its raw error rate is unacceptable for cryptographic correctness (which demands a Decryption Failure Rate below 2^{-128}), it is still exceptionally reliable compared to typical physical communication channels. WEAVER exploits this unused capacity by splitting the shared secret into two parts: the higher bits $\bar{\mu}$ and the lower bits $\dot{\mu}$. We encode $\bar{\mu}$ in the reliable MSB layer protected by a standard BCH code, and encode $\dot{\mu}$ in the noisier lower-bit channel protected by a concatenated code (a four-way repetition layer inner code plus a BCH outer code). This “woven” hierarchy allows us to pack more information into the same ring dimension, safely absorbing the massive noise induced by aggressive compression.

Layer 1: Most Significant Bits (MSB) Encoding. The first part of the message $\bar{\mu}$ is encoded into the most significant bits of the polynomial coefficients. Since the MSBs are relatively robust against the LWE error term, the error rate before correction is moderate. We employ a standard Bose-Chaudhuri-Hocquenghem (BCH) code (e.g., BCH1) to correct these residual errors, bringing the failure probability of this layer down to cryptographic insignificance.

Layer 2: Next-Most Significant Bits Encoding via 4-Way Repetition. For all parameter sets, the remaining message bits $\dot{\mu}$ are encoded into the next-most significant bits to embed sufficient internal message material without increasing the polynomial ring dimension n . However, these bits suffer from a significantly higher error rate due to the accumulated LWE noise and compression rounding errors. Simple scalar rounding is insufficient here. To overcome this, we utilize a **4-way repetition code** mapped across the polynomial coefficients. By spreading 1 bit of information across 4 coefficients (as seen in our `MsgEncode` algorithm), we achieve a substantial coding and processing gain against Gaussian-like noise. This spatial redundancy aggregates the noise variance across multiple coordinates, effectively smoothing out individual extreme noise spikes. To eliminate any remaining errors after the repetition decoding, we concatenate it with a secondary BCH code (BCH2). The synergy of the 4-way repetition code and BCH error correction allows WEAVER to extract reliable bits from highly noisy coefficients.

3.3 Parameter Selection Strategy

The parameter selection for WEAVER is formulated as a multi-objective optimization problem. Our primary goal is to minimize the total bandwidth (public key and ciphertext sizes) while strictly satisfying two constraints: (1) The Core-SVP hardness must meet or exceed the targeted NIST security categories (Levels 1, 3, and 5). (2) The average-case Decryption Failure Rate (DFR) must remain cryptographically negligible (e.g., $\leq 2^{-128}$).

To achieve this optimal balance, we decoupled the parameter selection into four distinct categories: algebraic structure, security scaling, bandwidth compression, and error correction.

3.3.1 Algebraic Parameters (n, q)

For the underlying polynomial ring $\mathcal{R}_q = \mathbb{Z}_q[X]/(X^n + 1)$, the ring dimension n and modulus q are selected per parameter set: WEAVER-640 uses $n = 128$ with $q = 3329$, WEAVER-1024 uses $n = 256$ with $q = 7681$, and WEAVER-2048 uses $n = 512$ with $q = 7681$. Both moduli are primes chosen to be NTT-friendly: $3329 \equiv 1 \pmod{256}$ and $7681 \equiv 1 \pmod{512}$, which enables an efficient Number Theoretic Transform (NTT) over the corresponding ring. The prime $q = 3329$

has been extensively vetted by the cryptographic community (e.g., in ML-KEM/Kyber) and enables optimized arithmetic implementations across hardware and software platforms. Keeping n as a power of 2 preserves the algebraic structure of the cyclotomic ring.

3.3.2 Security Scaling Parameters (k, η_1, η_2)

The module rank k serves as the primary coarse-grained knob for scaling the security level. We use $k = 5$ for Category 1 (with $n = 128$, hence module dimension $kn = 640$), and $k = 4$ for Categories 3 and 5 (with $n = 256$ and $n = 512$, respectively). To fine-tune the security and control the initial LWE noise, we utilize the Centered Binomial Distribution (CBD). The parameters η_1 (for secret keys) and η_2 (for randomness vectors) are chosen per parameter set, as listed in Table 2. A smaller η reduces the raw noise variance, leaving more “noise budget” for the compression step, while still providing sufficient entropy against combinatorial and algebraic attacks.

3.3.3 Bandwidth Compression Parameters (d_t, d_u, d_v)

The most distinctive aspect of WEAVER’s parameter strategy is the aggressive truncation of the public key and ciphertext components. We drop a significant number of lower bits, retaining only d_t, d_u , and d_v bits for $\mathbf{t}, \mathbf{u}$, and v , respectively. Unlike standard schemes that conservatively choose these values to avoid decryption failures, we intentionally push d_t, d_u, d_v to their extreme lower limits (e.g., $d_v = 6$ for WEAVER-640, and $d_v = 8$ for WEAVER-1024). This drastically reduces the serialized sizes. The massive rounding error introduced by this aggressive compression acts as an additional source of cryptographic noise, effectively amplifying the concrete security against lattice reduction attacks (as discussed in Section 4).

3.3.4 Error Correction Parameters (BCH and Repetition)

The aggressive compression strategy inevitably results in a raw noise variance that far exceeds the standard correctable threshold of pure rounding-based decoding. To resolve this, we parameterize our hierarchical error correction mechanism to perfectly absorb this excess noise.

- **Message Splitting:** For all parameter sets, the internal FO/PKE message is divided into higher bits ($\bar{\mu}$) and lower bits ($\hat{\mu}$); the lengths ℓ_1 and ℓ_2 are dynamically allocated based on the noise capacity of the specific security level. For WEAVER-640, we allocate $\ell_1 = 112$ and $\ell_2 = 16$.
- **4-fold repetition:** For the highly noisy lower bits, we utilize a 4-way repetition code. This spatial redundancy provides a significant processing gain, artificially lowering the raw bit error rate.
- **BCH Capacity (t_c):** We select BCH codes with an error-correcting capability of $t_c = 2$ to $t_c = 7$. This specific capacity was mathematically derived: it is the exact threshold required to pull the subgaussian tail of the residual errors down to $\leq 2^{-134}$, ensuring negligible average-case DFR without introducing excessive parity-check overhead.

By jointly optimizing these four sets of parameters, WEAVER achieves a favorable trade-off: it leverages heavy error correction to tolerate extreme compression, resulting in highly compact ciphertexts while maintaining robust security and correctness.

3.4 Decryption Failure Analysis

In this subsection, we analyze the decryption correctness of WEAVER in the honest-execution setting. Our goal is to justify the parameter choices and the use of the two-layer decoding mech-

anism by estimating the average-case decryption failure rate (DFR) under randomly generated keys, messages, and encryption randomness.

3.4.1 Layered Decoding Thresholds

The decoding procedure of WEAVER is hierarchical. As described in Algorithm 6, the lower-layer bits and upper-layer bits are recovered through different decision rules.

The purpose of the two-layer coding strategy is precisely to convert these moderate raw error probabilities into a negligible final DFR.

For the higher-bit layer, the BCH code corrects up to t_1 bit errors in each codeword. If the raw coefficient error probability is p_{high} , then the residual failure probability of this layer is bounded by the upper tail of the corresponding binomial distribution.

For the lower-bit layer, the 4-fold repetition first reduces the effective bit error probability, and the outer BCH code then corrects the remaining residual errors up to its correction capability t_2 . This produces a second-layer failure probability that is likewise negligible.

Thus, the decryption correctness should be analyzed separately for two-layer parameter sets:

- the higher-bit layer (Layer 1) protected by BCH decoding, and
- the lower-bit layer (Layer 2) protected by 4-fold repetition followed by BCH decoding.

3.4.2 Single Bit Failure Rate

We first state the following lemmas about single bit failure rate (BFR) at each layer. Then we will use them to deduce the overall decryption failure rate (DFR) of our scheme.

According to Algorithm 5, the receiver computes

$$w' = v - \text{NTT}^{-1}(\hat{\mathbf{s}}^T \circ \text{NTT}(\mathbf{u})). \quad (1)$$

Let $\mathbf{c}_t$, $\mathbf{c}_u$, and c_v denote the distortions introduced by compression and decompression. Substituting the encryption equations from Algorithm 2, the raw decryption noise polynomial $e_{\text{raw}} = w' - w$ can be written as

$$e_{\text{raw}} = \mathbf{c}_t^T \mathbf{r} - \mathbf{s}^T \mathbf{c}_u + c_v. \quad (2)$$

Before looking into the details, we first define some error distributions needed in our failure rate analysis. First we define:

Definition 3.1 (Compression Error). *Let ψ_d^k be the following distribution over $\mathcal{R}_q^k$:*

1. Choose uniformly random $\mathbf{x} \leftarrow \mathcal{R}_{q^+}^k$
2. return $(\text{Decompress}_q(\text{Compress}_q(\mathbf{x}, d), d) - \mathbf{x}) \bmod^{\pm} q$.

Definition 3.2 (Randomized Lifting Error). *Let φ_d^k be the following distribution over $\mathcal{R}_q^k$:*

1. Choose uniformly random $\mathbf{x} \leftarrow \mathcal{R}_{q^+}^k$
2. Choose uniformly random $\mathbf{e} \leftarrow \text{Preim}_q^{\pm}(\mathbf{x}, d)$
3. return $\text{Inv}_q(\text{Compress}_q(\mathbf{x}, d), d) - \mathbf{x} \bmod^{\pm} q$, which is equivalent to returning $\text{Decompress}_q(\text{Compress}_q(\mathbf{x}, d), d) - \mathbf{x} + \mathbf{e} \bmod^{\pm} q$.

The higher-bit layer message embedding has followed the common practice, namely we have:

Lemma 3.3 (Layer 1 BFR; modified from [11]). *Let k be a positive integer, q be an odd positive integer. Let $\mathbf{s}, \mathbf{r}$ be random variables that have the same distribution as in Algorithms 1 and 2. Also, let $\mathbf{c}_t \leftarrow \varphi_{d_t}^k$, $\mathbf{c}_u \leftarrow \psi_{d_u}^k$, $c_v \leftarrow \psi_{d_v}^k$. Denote: $\delta_0^{(1)} = \Pr[|(\mathbf{c}_t^T \mathbf{r} - \mathbf{s}^T \mathbf{c}_u + c_v)_i| \geq \lfloor q/4 \rfloor]$, then the **single bit failure rate** at layer 1 is $\delta_0^{(1)}$.*

Proof. Let the public-key compression/lifting error, the ciphertext-vector compression error, and the second-component compression error be written as

$$\mathbf{t}_{\text{lift}} = \mathbf{t} + \mathbf{c}_t, \quad \mathbf{u}_{\text{dec}} = \mathbf{u} + \mathbf{c}_u, \quad v_{\text{dec}} = v_0 + w + c_v,$$

where $\mathbf{c}_t \leftarrow \varphi_{d_t}^k$, $\mathbf{c}_u \leftarrow \psi_{d_u}^k$, and $c_v \leftarrow \psi_{d_v}^k$ denote the corresponding coefficient-wise error distributions as in Definition 3.1 and 3.2. In an honest encryption we have $\mathbf{t} = \mathbf{A}\mathbf{s}$, $\mathbf{u} = \mathbf{A}^T \mathbf{r}$, and $v_0 = \mathbf{t}_{\text{lift}}^T \mathbf{r}$. Hence the value computed by decryption satisfies

$$\begin{aligned} w' &= v_{\text{dec}} - \mathbf{s}^T \mathbf{u}_{\text{dec}} \\ &= v_0 + w + c_v - \mathbf{s}^T (\mathbf{A}^T \mathbf{r} + \mathbf{c}_u) \\ &= (\mathbf{t} + \mathbf{c}_t)^T \mathbf{r} + w + c_v - \mathbf{s}^T \mathbf{A}^T \mathbf{r} - \mathbf{s}^T \mathbf{c}_u \\ &= w + \mathbf{c}_t^T \mathbf{r} - \mathbf{s}^T \mathbf{c}_u + c_v, \end{aligned}$$

where the last equality uses $\mathbf{t} = \mathbf{A}\mathbf{s}$, and therefore $\mathbf{t}^T \mathbf{r} = \mathbf{s}^T \mathbf{A}^T \mathbf{r}$. Thus the raw coefficient error is

$$e_{\text{raw}}[i] = (\mathbf{c}_t^T \mathbf{r} - \mathbf{s}^T \mathbf{c}_u + c_v)_i.$$

Assume now that the lower-layer word has been decoded correctly and its contribution has been subtracted, as in Algorithm 6. The remaining high-layer coefficient has the form

$$\bar{w}_i = \frac{q-1}{2} \tilde{\mu}_i + e_{\text{raw}}[i] \quad (\tilde{\mu}_i \in \{0, 1\}).$$

Therefore a high-layer coefficient can be decoded incorrectly only when $|e_{\text{raw}}[i]| \geq \lfloor q/4 \rfloor$. Taking the probability over $\mathbf{s}, \mathbf{r}$, and the compression/lifting error distributions gives

$$\Pr[\text{Layer 1 bit } i \text{ fails}] = \Pr[|(\mathbf{c}_t^T \mathbf{r} - \mathbf{s}^T \mathbf{c}_u + c_v)_i| \geq \lfloor q/4 \rfloor].$$

□

For the lower-bit layer, the message is scaled by $\lfloor q/4 \rfloor$ and decoded after reduction modulo $(q-1)/2$. Although the raw scalar decision margin for this layer is inherently narrower before error correction, each lower-layer bit is distributed across four distant coefficients using a 4-way repetition code with soft-decision decoding. This multi-dimensional repetition effectively elevates the decision threshold to B_{dec} , significantly boosting the scheme's robustness against heavy rounding and decryption noise.

Lemma 3.4 (Layer 2 BFR). *Let all symbols be the same as in the previous lemma. Let $B_{\text{dec}} = (4 \cdot \lfloor q/4 \rfloor)/2 = (q-1)/2$, and let the i -th coefficient of the decryption error polynomial be $e_{\text{raw}}[i] = (\mathbf{c}_t^T \mathbf{r} - \mathbf{s}^T \mathbf{c}_u + c_v)_i$, $stp = 32/64/128$ for WEAVER-640, WEAVER-1024 and WEAVER-2048 respectively. Denote:*

$$\delta_0^{(2)} = \Pr \left[|e_{\text{raw}}[i]| + |e_{\text{raw}}[i + stp]| + |e_{\text{raw}}[i + 2 \cdot stp]| + |e_{\text{raw}}[i + 3 \cdot stp]| \geq B_{\text{dec}} \right],$$

*then the **single bit failure rate after repetition decoding** at layer 2 is $\delta_0^{(2)}$.*

Proof. The derivation of the raw error is identical to the Layer 1 case, the decrypted coefficient differs from the encoded message coefficient by $e_{\text{raw}}[j] = (\mathbf{c}_t^T \mathbf{r} - \mathbf{s}^T \mathbf{c}_u + c_v)_j$. Thus only the lower-layer decision rule has to be considered.

Let $a = \lfloor q/4 \rfloor$ and $h = (q-1)/2 = 2a$. For a lower-layer code bit $\ddot{\mu}_i$, Algorithm 6 removes the high-layer contribution by reducing modulo h , and aggregates the four repeated positions using

$$ee_i = \sum_{r=0}^3 |(a\ddot{\mu}_i + e_{\text{raw}}[i + r \cdot \text{stp}]) \bmod^+ h - a|.$$

The decoder outputs one iff $ee_i < h$. If $\ddot{\mu}_i = 1$, then $ee_i \leq \sum_{r=0}^3 |e_{\text{raw}}[i + r \cdot \text{stp}]|$. If $\ddot{\mu}_i = 0$, then each term has distance at least $a - |e_{\text{raw}}[i + r \cdot \text{stp}]|$ from the center a , and therefore

$$ee_i \geq 4a - \sum_{r=0}^3 |e_{\text{raw}}[i + r \cdot \text{stp}]| = 2h - \sum_{r=0}^3 |e_{\text{raw}}[i + r \cdot \text{stp}]|.$$

Consequently, whenever $\sum_{r=0}^3 |e_{\text{raw}}[i + r \cdot \text{stp}]| < h$, the decoder outputs the correct lower-layer bit in both cases. Thus a lower-layer bit can fail only if

$$\sum_{r=0}^3 |e_{\text{raw}}[i + r \cdot \text{stp}]| \geq h = B_{\text{dec}}.$$

Taking the probability over the same randomness and error distributions gives

$$\Pr[\text{Layer 2 bit } i \text{ fails}] = \Pr \left[\sum_{r=0}^3 |e_{\text{raw}}[i + r \cdot \text{stp}]| \geq B_{\text{dec}} \right].$$

□

3.4.3 Independence Assumption and Overall Decryption Failure Rate

Proposition 3.5. *For a middle-result bit string of length ℓ with BFR δ_0 , if δ_0 is small enough, the estimation of its DFR δ after corrects up to t -bits error (i.e. the number of flipped bits in ℓ -bits string does not exceed t) can be done by applying “independence assumption”, which we claim to achieve enough precision:*

$$\delta = 1 - \sum_{i=0}^t \binom{\ell}{i} \cdot \delta_0^i (1 - \delta_0)^{\ell-i} = \text{binom.sf}(t, \ell, \delta_0).$$

where *binom.sf* is the survival function of binomial distribution.

Theorem 3.6 (Layer 2 DFR). *Suppose layer 2 BFR is $\delta_0^{(2)}$, then after applying a binary error correction codes with codeword length ℓ_2 and error tolerance t_2 , the overall DFR in layer 2 is estimated by:*

$$\delta^{(2)} = \text{binom.sf}(t_2, \ell_2, \delta_0^{(2)}).$$

Theorem 3.7 (Layer 1 and Overall DFR). *Suppose layer 1 BFR is $\delta_0^{(1)}$ and layer 2 DFR is $\delta^{(2)}$, then, with exception of probability $\delta^{(2)}$: After applying a binary error correction codes with codeword length ℓ_1 and error tolerance t_1 , the overall DFR in layer 1 is estimated by:*

$$\delta^{(1)} = \text{binom.sf}(t_1, \ell_1, \delta_0^{(1)}).$$

Therefore, the overall DFR of our scheme can be upper bounded by (union bound): $\delta = \delta^{(1)} + \delta^{(2)}$.

Proof. In the decapsulated ring element w' , each coefficient consists of three contributions: the high-bit component of the plaintext, the low-bit component, and the accumulated error. Formally,

$$w'_i = \frac{q-1}{2} \cdot \tilde{\mu}_i + \left\lfloor \frac{q}{4} \right\rfloor \cdot \ddot{\mu}_i + e_{\text{raw}}[i]$$

where the subscript i denotes the i -th coefficient (or bit), for $i = 0, \dots, n-1$.

In the message decoding procedure (Algorithm 6), lines 11–16 remove the low-bit component $\lfloor q/4 \rfloor \cdot \ddot{\mu}_i$ from w'_i , assuming that each $\ddot{\mu}_i$ is correctly recovered (with probability $1 - \delta^{(2)}$). The remaining term is

$$\bar{w}_i = \frac{q-1}{2} \cdot \tilde{\mu}_i + e_{\text{raw}}[i]$$

which enables the recovery of the high bits $\tilde{\mu}_i$ without interference from the low bits. $\square$

3.5 Rationale for Symmetric Primitives

The hash functions H, G, J , the seed-derivation extendable-output function XOF , the public matrix-expansion function XOF_{pub} , and the pseudorandom function PRF are required to meet the security parameters specified in Section 2.4. Concrete implementations may instantiate these components with suitable hash/ XOF / PRF algorithms that meet the corresponding classical and quantum security requirements; if a particular standardized primitive does not meet a required level, it can be replaced by a stronger primitive with the same interface. In contrast, the matrix-expansion function XOF_{pub} only needs to produce a pseudorandom (near-uniform) output stream, since both its seed and the resulting matrix are public; it may therefore be instantiated by a fixed-security XOF such as SHAKE128 at all security levels.

The rationale for this modular choice is threefold:

- **Security Parameterization:** The security requirements of the symmetric components are explicitly tied to the target parameter set, rather than to a fixed hash-function instantiation.
- **Domain Separation:** The matrix-expansion function XOF_{pub} and the seed-derivation function XOF allow for clean domain separation when expanding the public matrix and sampling the secret vectors, ensuring that the generated pseudorandomness is statistically independent.
- **Implementation Flexibility:** The specification permits replacing the underlying hash/ XOF / PRF algorithms by stronger or future standardized primitives while preserving the same algorithmic interfaces.

3.5.1 Implicit Rejection FO Transform

To lift the IND-CPA secure PKE to an IND-CCA secure KEM, WEAVER employs a variant of the FO transform with *implicit rejection*.

In traditional explicit rejection schemes, decapsulation outputs a dedicated error symbol when ciphertext verification fails. This approach is notoriously difficult to implement in constant time, as the early abort or distinct error handling often leaks side-channel information, leading to practical attacks (e.g., Bleichenbacher-style attacks).

By contrast, our implicit rejection mechanism (Algorithm 9) ensures that upon a verification failure, the decapsulation algorithm computes a pseudorandom shared secret using a secret random value z stored in the KEM secret key. From the adversary’s perspective, this output is computationally indistinguishable from a valid shared secret. This design choice fundamentally decouples the execution time and the output distribution from the validity of the ciphertext, significantly reducing the attack surface for timing and power-analysis side-channels.

4 Security Statement and Analysis

4.1 Theoretical Security

In this section, we analyze the IND-CPA security of the underlying public-key encryption scheme of WEAVER-Inv. A central complication in standard bandwidth-optimized lattice schemes is that the public key is aggressively compressed. Consequently, the decompressed public key used during encryption is not uniformly distributed, forcing security proofs to either rely on non-standard, compression-aware assumptions or to absorb the compression loss into the error term, which complicates the analysis.

Our scheme addresses this issue by employing the randomized lifting mechanism Inv_q during encryption. Rather than using a deterministic decompression of the compressed public key, the encryptor samples a uniformly random preimage from the corresponding compression bucket. This yields a joint-distribution property that restores the correct uncompressed view needed in the security reduction.

4.1.1 Hardness Assumptions

We first recall the standard decisional Module Learning with Rounding assumption. Throughout, $\text{MLWR}_{k,\ell,\eta,d}$ denotes the problem where the public matrix has ℓ rows and k columns (i.e., the secret vector has dimension k and the output has dimension ℓ).

Definition 4.1 (Decisional MLWR Assumption). *Let $k, \ell \in \mathbb{N}$, let η be the parameter of the centered binomial distribution, and let d be a compression precision. The decisional $\text{MLWR}_{k,\ell,\eta,d}$ problem is to distinguish between the following two distributions:*

- **Real:** sample $\mathbf{A} \leftarrow \$\mathcal{R}_q^{\ell \times k}$ and $\mathbf{s} \leftarrow \text{CBD}_\eta^k$, and output $(\mathbf{A}, \text{Compress}_q(\mathbf{A}\mathbf{s}, d))$.
- **Random:** sample $\mathbf{A} \leftarrow \$\mathcal{R}_q^{\ell \times k}$ and $\mathbf{u} \leftarrow \$\mathcal{R}_q^\ell$, and output $(\mathbf{A}, \text{Compress}_q(\mathbf{u}, d))$.

For any adversary $\mathcal{B}$, its distinguishing advantage is denoted $\text{Adv}_{k,\ell,\eta,d}^{\text{MLWR}}(\mathcal{B})$.

We next define the shifted variant needed for the second ciphertext component.

Definition 4.2 (Decisional Shifted-MLWR Assumption). *Let $k, \ell \in \mathbb{N}$ with $k \leq \ell$, let η be the parameter of the centered binomial distribution, let d_1 and d_2 be compression precisions, and let $\Delta \in \mathcal{R}_q^\ell$ be a public offset. For any vector $\mathbf{x} \in \mathcal{R}_q^\ell$, write $\mathbf{x} = (\mathbf{x}_{\leq k}, \mathbf{x}_{>k})$ with $\mathbf{x}_{\leq k} \in \mathcal{R}_q^k$ and $\mathbf{x}_{>k} \in \mathcal{R}_q^{\ell-k}$, and define*

$$\text{Compress}_{q,d_1,d_2}(\mathbf{x}) := (\text{Compress}_q(\mathbf{x}_{\leq k}, d_1), \text{Compress}_q(\mathbf{x}_{>k}, d_2)).$$

The decisional Shifted-MLWR $_{k,\ell,\eta,(d_1,d_2),\Delta}$ problem is to distinguish between the following two distributions:

- **Real:** sample $\mathbf{A} \leftarrow \$\mathcal{R}_q^{\ell \times k}$ and $\mathbf{s} \leftarrow \text{CBD}_\eta^k$, and output $(\mathbf{A}, \text{Compress}_{q,d_1,d_2}(\mathbf{A}\mathbf{s} + \Delta))$.
- **Random:** sample $\mathbf{A} \leftarrow \$\mathcal{R}_q^{\ell \times k}$ and $\mathbf{u} \leftarrow \$\mathcal{R}_q^\ell$, and output $(\mathbf{A}, \text{Compress}_{q,d_1,d_2}(\mathbf{u} + \Delta))$.

For any adversary $\mathcal{B}$, its distinguishing advantage is denoted $\text{Adv}_{k,\ell,\eta,(d_1,d_2),\Delta}^{\text{Shifted-MLWR}}(\mathcal{B})$.

Remark 4.3. Since $\mathbf{u}$ is uniform over $\mathcal{R}_q^\ell$ and Δ is public and fixed, the random distribution in Definition 4.2 is identical to

$$(\mathbf{A}, \text{Compress}_{q,d_1,d_2}(\mathbf{u})).$$

Thus the shifted assumption differs from standard MLWR only in the real distribution, where the public offset is added before the final component-wise compression.

A technical ingredient in the proof is the following joint-distribution property of randomized lifting.

Lemma 4.4 (Joint Distribution of Randomized Lifting). *Let $\mathbf{U} \leftarrow \$\mathcal{R}_q^k$, let $\mathbf{Y} := \text{Compress}_q(\mathbf{U}, d_t)$, and let $\mathbf{U}' \leftarrow \text{Inv}_q(\mathbf{Y}, d_t)$, where Inv_q samples uniformly from the coefficient-wise preimage bucket. Then the joint distribution of $(\mathbf{Y}, \mathbf{U}')$ is identical to the joint distribution of $(\text{Compress}_q(\mathbf{U}, d_t), \mathbf{U})$. Equivalently,*

$$(\mathbf{Y}, \mathbf{U}') \equiv (\text{Compress}_q(\mathbf{U}, d_t), \mathbf{U}).$$

Proof. It suffices to prove the claim coefficient-wise. Fix any $a \in \mathbb{Z}_q$ and $y \in \mathbb{Z}_{2^{d_t}}$. If $\text{Compress}_q(a, d_t) \neq y$, then both joint probabilities are zero. Otherwise,

$$\Pr[Y = y, U' = a] = \Pr[Y = y] \cdot \Pr[U' = a \mid Y = y].$$

Since $Y = \text{Compress}_q(U, d_t)$ with $U \leftarrow \$\mathbb{Z}_q$, we have

$$\Pr[Y = y] = \frac{|\text{Preim}_q(y, d_t)|}{q},$$

and since $\text{Inv}_q(y, d_t)$ samples uniformly from $\text{Preim}_q(y, d_t)$,

$$\Pr[U' = a \mid Y = y] = \frac{1}{|\text{Preim}_q(y, d_t)|}.$$

Hence

$$\Pr[Y = y, U' = a] = \frac{1}{q}.$$

On the other hand,

$$\Pr[\text{Compress}_q(U, d_t) = y, U = a] = \Pr[U = a] = \frac{1}{q},$$

whenever $\text{Compress}_q(a, d_t) = y$. Therefore the joint distributions are identical. Applying this argument independently coefficient-wise proves the vector case. $\square$

Theorem 4.5 (IND-CPA Security of WEAVER PKE). *Let WEAVER.CPAPKE be the public-key encryption scheme described in Algorithms 1 and 2. For any quantum polynomial-time adversary $\mathcal{A}$, there exist quantum polynomial-time adversaries $\mathcal{B}_1$ and $\mathcal{B}_2$ such that*

$$\text{Adv}_{\text{WEAVER}}^{\text{IND-CPA}}(\mathcal{A}) \leq \text{Adv}_{k, k, \eta_1, d_t}^{\text{MLWR}}(\mathcal{B}_1) + \text{Adv}_{k, k+1, \eta_2, (d_u, d_v), \Delta_b}^{\text{Shifted-MLWR}}(\mathcal{B}_2),$$

where

$$\Delta_b = \begin{pmatrix} \mathbf{0} \\ M_b \end{pmatrix} \in \mathcal{R}_q^{k+1}, \quad M_b = \text{MsgEncode}(\mu_b) \in \mathcal{R}_q.$$

Proof. We prove the theorem through a sequence of games. For each i , let S_i denote the event that $\mathcal{A}$ correctly guesses the hidden challenge bit in Game i .

Game 0 (Real Experiment). This is the real IND-CPA experiment. The challenger runs the genuine key-generation algorithm, generating $\mathbf{A}$ and $\mathbf{s} \leftarrow \text{CBD}_{\eta_1}^k$, and sets

$$pk = \text{Compress}_q(\mathbf{A}\mathbf{s}, d_t).$$

Upon receiving two challenge messages μ_0, μ_1 , it picks $b \leftarrow \$\{0, 1\}$ and encrypts μ_b honestly using fresh coins seed_1 . By definition,

$$\text{Adv}_{\text{WEAVER}}^{\text{IND-CPA}}(\mathcal{A}) = \left| \Pr[S_0] - \frac{1}{2} \right|.$$

Game 1 (Replace Public Key with Uniform). We modify key generation. Instead of computing $\mathbf{t} = \mathbf{A}\mathbf{s}$, the challenger samples $\mathbf{b} \leftarrow \$\mathcal{R}_q^k$ uniformly at random and sets $pk := \text{Compress}_q(\mathbf{b}, d_t)$. All other steps remain unchanged.

The public-key distribution in Game 0 is $(\mathbf{A}, \text{Compress}_q(\mathbf{A}\mathbf{s}, d_t))$, whereas in Game 1 it is $(\mathbf{A}, \text{Compress}_q(\mathbf{b}, d_t))$. Any distinguisher between Game 0 and Game 1 therefore yields a distinguisher for $\text{MLWR}_{k,k,\eta_1,d_t}$. Hence there exists a quantum polynomial-time adversary $\mathcal{B}_1$ such that

$$|\Pr[S_1] - \Pr[S_0]| \leq \text{Adv}_{k,k,\eta_1,d_t}^{\text{MLWR}}(\mathcal{B}_1).$$

Game 2 (Replace Randomized Lift by a Fresh Uniform Lift). In Game 1, during encryption the challenger computes the lifted public key as

$$\mathbf{t}_{\text{lift}} \leftarrow \text{Inv}_q(pk, d_t; \text{PRF}(\text{seed}_1, \text{ctr}_0)),$$

and the ephemeral secret as

$$\mathbf{r}[i] \leftarrow \text{CBD}_{\eta_2}(\text{PRF}(\text{seed}_1, \text{ctr}_i)), \quad i = 1, \dots, k,$$

where the counter values $\text{ctr}_0 = 0$ and $\text{ctr}_1, \dots, \text{ctr}_k \geq 1$ are distinct. Since the PRF outputs on distinct inputs are computationally independent, the lifting randomness and the randomness used to sample $\mathbf{r}$ may be treated as independent.

Now in Game 1 we have $pk = \text{Compress}_q(\mathbf{b}, d_t)$ for a uniformly random $\mathbf{b} \in \mathcal{R}_q^k$. By Lemma 4.4, the joint distribution of

$$(pk, \mathbf{t}_{\text{lift}}) = (\text{Compress}_q(\mathbf{b}, d_t), \text{Inv}_q(\text{Compress}_q(\mathbf{b}, d_t), d_t))$$

is identical to the joint distribution of

$$(\text{Compress}_q(\mathbf{T}, d_t), \mathbf{T}), \quad \mathbf{T} \leftarrow \$\mathcal{R}_q^k.$$

Therefore, without changing the joint distribution seen by $\mathcal{A}$, we may equivalently sample a fresh uniform vector $\mathbf{T} \leftarrow \$\mathcal{R}_q^k$, set $pk := \text{Compress}_q(\mathbf{T}, d_t)$, and use $\mathbf{T}$ directly in place of $\mathbf{t}_{\text{lift}}$ during encryption. Hence

$$\Pr[S_2] = \Pr[S_1].$$

Game 3 (Replace the Challenge Ciphertext via Shifted-MLWR). In Game 2, the uncompressed ciphertext components are

$$\mathbf{u} = \mathbf{A}^T \mathbf{r}, \quad v_0 = \mathbf{T}^T \mathbf{r},$$

and the challenge ciphertext is

$$c_1 = \text{Compress}_q(\mathbf{u}, d_u), \quad c_2 = \text{Compress}_q(v_0 + M_b, d_v),$$

where $\mathbf{r} \leftarrow \text{CBD}_{\eta_2}^k$ and

$$M_b = \text{MsgEncode}(\mu_b) \in \mathcal{R}_q.$$

Define the stacked matrix

$$\mathbf{A}^* = \begin{pmatrix} \mathbf{A}^T \\ \mathbf{T}^T \end{pmatrix} \in \mathcal{R}_q^{(k+1) \times k},$$

and the public offset

$$\Delta_b = \begin{pmatrix} \mathbf{0} \\ M_b \end{pmatrix} \in \mathcal{R}_q^{k+1}.$$

Since $\mathbf{A}$ is sampled uniformly at random, $\mathbf{A}^T$ is also uniform over $\mathcal{R}_q^{k \times k}$. Moreover, $\mathbf{T}$ is an independent uniform vector in $\mathcal{R}_q^k$, so $\mathbf{A}^*$ is uniformly distributed over $\mathcal{R}_q^{(k+1) \times k}$.

The pair (c_1, c_2) is precisely the compression of

$$\mathbf{A}^* \mathbf{r} + \Delta_b = \begin{pmatrix} \mathbf{A}^T \mathbf{r} \\ \mathbf{T}^T \mathbf{r} + M_b \end{pmatrix},$$

with compression precisions (d_u, d_v) , namely d_u on the first k coordinates and d_v on the last coordinate, respectively. Therefore the challenge ciphertext distribution in Game 2 is an instance of $\text{Shifted-MLWR}_{k, k+1, \eta_2, (d_u, d_v), \Delta_b}$.

In Game 3, the challenger replaces the challenge ciphertext with

$$c_1 := \text{Compress}_q(\mathbf{u}^*, d_u), \quad c_2 := \text{Compress}_q(v^* + M_b, d_v),$$

where $\mathbf{u}^* \leftarrow \$\mathcal{R}_q^k$ and $v^* \leftarrow \$\mathcal{R}_q$ are uniformly random. Hence there exists a quantum polynomial-time adversary $\mathcal{B}_2$ such that

$$|\Pr[S_3] - \Pr[S_2]| \leq \text{Adv}_{k, k+1, \eta_2, (d_u, d_v), \Delta_b}^{\text{Shifted-MLWR}}(\mathcal{B}_2).$$

Game 4 (Challenge Ciphertext is Independent of b). This game is identical to Game 3; we introduce it only to record the following observation. Since v^* is uniformly distributed over $\mathcal{R}_q$, for every fixed message offset $M_b \in \mathcal{R}_q$, the shifted value $v^* + M_b$ is also uniformly distributed over $\mathcal{R}_q$. Therefore the distribution of $c_2 = \text{Compress}_q(v^* + M_b, d_v)$ is independent of the hidden bit b . Likewise, $c_1 = \text{Compress}_q(\mathbf{u}^*, d_u)$ is already independent of b . Hence the entire challenge ciphertext carries no information about b , and

$$\Pr[S_4] = \frac{1}{2}.$$

Since Game 4 is identical to Game 3, we also have $\Pr[S_4] = \Pr[S_3]$.

Conclusion. By the triangle inequality applied to the full game sequence $0 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 4$,

$$\begin{aligned} |\Pr[S_0] - \tfrac{1}{2}| &= |\Pr[S_0] - \Pr[S_4]| \\ &\leq |\Pr[S_0] - \Pr[S_1]| + |\Pr[S_1] - \Pr[S_2]| + |\Pr[S_2] - \Pr[S_3]| + |\Pr[S_3] - \Pr[S_4]|. \end{aligned}$$

Substituting $\Pr[S_2] = \Pr[S_1]$ (Game 2), $\Pr[S_4] = \Pr[S_3]$ (Game 4 is identical to Game 3), and the bounds from Game 1 and Game 3 yields

$$\text{Adv}_{\text{WEAVER}}^{\text{IND-CPA}}(\mathcal{A}) \leq \text{Adv}_{k, k, \eta_1, d_t}^{\text{MLWR}}(\mathcal{B}_1) + \text{Adv}_{k, k+1, \eta_2, (d_u, d_v), \Delta_b}^{\text{Shifted-MLWR}}(\mathcal{B}_2).$$

This completes the proof. $\square$

4.1.2 From IND-CPA PKE to IND-CCA KEM

We now explain how the above IND-CPA security result for the underlying PKE implies IND-CCA security of the KEM obtained from it via the Fujisaki-Okamoto (FO) transform with implicit rejection.

The KEM construction of WEAVER is given in Algorithms 6, 7, and 8. During encapsulation, a random message m is sampled and the hash function G is used to derive both the shared secret K and the deterministic encryption coins seed_1 . During decapsulation, the receiver decrypts the ciphertext to obtain m' , recomputes $(K', \text{seed}'_1) = G(m' \| H(pk))$, and re-encrypts m' using seed'_1 to verify ciphertext consistency. If the verification fails, the decapsulation algorithm returns the implicit-rejection value $J(z, ct)$ instead of aborting.

This is the standard FO paradigm with implicit rejection. It is structurally compatible with WEAVING because decapsulation is deterministic: the receiver uses the canonical decompression map Decompress_q in $\text{WEAVING.CPAPKE.Dec}$, so the re-encryption check is well defined. The residual possibility of honest decryption failure is handled separately through the negligible-failure assumptions below.

Theorem 4.6 (IND-CCA Security of WEAVING KEM). *Assume that the hash functions H , G , and J are modeled as random oracles. Assume furthermore that:*

1. *the underlying PKE scheme WEAVING.CPAPKE satisfies the IND-CPA security bound of Theorem 4.5;*
2. *the honest-execution decapsulation failure probability is δ , as analyzed in Section 3.4;*
3. *the strongest known precomputation-based failure-boosting attack still yields only negligible success probability within the target query budget, as analyzed in Section 4.3.*

Then for any quantum polynomial-time adversary $\mathcal{A}$ making at most q_D decapsulation queries and at most q_H, q_G random-oracle queries, there exists a quantum polynomial-time adversary $\mathcal{B}$ such that

$$\text{Adv}_{\text{WEAVING-KEM}}^{\text{IND-CCA}}(\mathcal{A}) \leq 2 \cdot \text{Adv}_{\text{WEAVING-PKE}}^{\text{IND-CPA}}(\mathcal{B}) + \frac{q_H^2 + q_G^2}{|\mathcal{M}|} + q_D \cdot \delta,$$

where $|\mathcal{M}|$ is the message space size and δ is the honest-execution decapsulation failure probability.

Proof sketch. The proof follows the standard game-hopping argument for the Fujisaki–Okamoto transform with implicit rejection (cf. [23, 27]).

Step 1: Random-oracle programming. Starting from the real IND-CCA experiment, one first programs the random oracles so that the challenge encapsulation is represented in the standard FO form, where the challenge ciphertext is generated by deterministic encryption under coins derived from the hidden message. Any adversary that distinguishes this reformulation from the real experiment can be turned into an adversary against the random-oracle abstraction, contributing the $(q_H^2 + q_G^2)/|\mathcal{M}|$ term.

Step 2: Replace challenge ciphertext via IND-CPA. One then replaces the challenge ciphertext with a ciphertext obtained from the IND-CPA challenger for WEAVING.CPAPKE . By Theorem 4.5, this modification is computationally indistinguishable under the MLWR and Shifted-MLWR assumptions, contributing the $2 \cdot \text{Adv}^{\text{IND-CPA}}$ term.

Step 3: Handle decapsulation queries. It remains to analyze decapsulation queries.

- **Invalid ciphertexts.** If a queried ciphertext fails the re-encryption check, then by construction decapsulation returns the pseudorandom value $J(z, ct)$, which is independent of the challenge secret.
- **Valid ciphertexts (honest execution).** If a queried ciphertext passes the re-encryption check and is a fresh valid ciphertext, the standard FO argument applies.
- **Failure-exploiting ciphertexts.** If a queried ciphertext passes the re-encryption check by exploiting a decryption-failure event, the probability of this event is bounded by δ per query. Over q_D decapsulation queries, the union bound gives a total contribution of $q_D \cdot \delta$, which is negligible by assumption (2). The stronger precomputation-based failure-boosting attack is further analyzed in Section 4.3, which shows that even adversarially searched ciphertexts do not raise the exploitable failure probability beyond a negligible level within the relevant query budget, consistent with assumption (3).

Combining all three steps establishes the claimed bound. $\square$

4.2 Concrete Security Estimates

Concrete Hardness Methodology for Shifted-MLWR. To estimate the concrete security of WEAVER, we model our Shifted-MLWR instances as equivalent MLWE instances where the rounding error is treated as a uniformly random additive noise.

We use this methodology as a concrete-estimation model rather than as a black-box reduction from ordinary MLWR. Indeed, given only a rounded value $\text{Compress}_q(x, d)$, one cannot in general compute $\text{Compress}_q(x + \Delta, d)$ for an arbitrary public offset Δ . Therefore, we treat Shifted-MLWR as the shifted, two-precision variant of MLWR that appears naturally in the ciphertext distribution. Its use is justified by the shift-stability argument in Section A: in the Liu–Wang [32] LWE-to-LWR proof path, statistical closeness is established before the final rounding step, and the public translation followed by the coordinate-wise rounding with precisions (d_u, d_v) is a deterministic post-processing that cannot increase statistical distance. Under this interpretation, the public shift introduces no additional secret-dependent algebraic information beyond the same two-precision rounding structure already accounted for in the concrete estimates.

Specifically, for a compression parameter $d \in \{d_t, d_u, d_v\}$, the rounding operation introduces an error e_{round} that we model as being uniformly distributed in the interval $[-\frac{q}{2^{d+1}}, \frac{q}{2^{d+1}}]$, with variance $\sigma_{\text{round}}^2 \approx \frac{q^2}{12 \cdot 2^{2d}}$. We then feed these effective noise variances into state-of-the-art lattice estimators to evaluate the concrete cost of primal and dual attacks.

We justify the soundness of this methodology for the *shifted* variant under two key observations:

- **Geometric Invariance under Public Shift:** The public offset Δ is a known, fixed vector that is completely independent of the secret $\mathbf{s}$. In the underlying lattice $\mathcal{L} = \{\mathbf{y} \in \mathbb{Z}_q^n \mid \mathbf{y} \equiv \mathbf{A}\mathbf{s} \pmod{q}\}$, adding Δ merely translates the entire target space by a constant vector. This translation does not alter the shortest vector structure, the volume of the fundamental domain, or any other geometric invariants of the lattice. Thus, the difficulty of the bounded distance decoding (BDD) search space remains unchanged.
- **Translation Invariance of Uniform Noise:** In the random branch, translation by the fixed public offset Δ is a bijection on $\mathcal{R}_q^\ell$, so $\mathbf{u} + \Delta$ is uniform whenever $\mathbf{u}$ is uniform. In the real branch, the shifted rounding map is handled through the proof-path stability argument above: the public shift is applied only as part of the final deterministic post-processing. Accordingly, the concrete estimator uses the same rounding-noise variances as in the corresponding unshifted two-precision instance.

Therefore, evaluating the concrete hardness of Shifted-MLWR by treating the shifted rounding error as a standard uniform additive error of variance σ_{round}^2 should be understood as the concrete attack-estimation methodology used for the instantiated two-precision rounded lattice instances. This methodology is consistent with the shift-stability result above and with the standard cryptanalytic treatment of rounding-based schemes.

Key Recovery as Search-MLWR. When estimating the concrete cost of key recovery in WEAVER (i.e., solving the underlying Search-MLWR instances), the adversary can choose to attack those defined by the public key—recovering the long-term secret $\mathbf{s}$ (see Algorithm 1)—or those defined by the ciphertext c_1 and c_2 —recovering the ephemeral secret $\mathbf{r}$ (see Algorithm 2). Since $d_t = d_u$ for all parameter sets (see Table 2), the rounding error introduced by compression is identical for the public-key-based instance (using d_t) and the c_1 -based instance (using d_u); the only difference lies in the CBD parameters η_1 and η_2 , which govern the norm of the secret vector.

For WEAVER-1024 and WEAVER-2048 we have $\eta_1 = \eta_2$, so the rounding noise and the secret standard deviation are identical between the two settings. For WEAVER-640, the asymmetric choice $\eta_1 = 3$, $\eta_2 = 2$ gives a smaller standard deviation for the secret in the ciphertext-derived instance, further reducing its cost. We note that the second ciphertext component c_2 (Algorithm 2, Line 21) does not constitute a strict MLWR equation for $\mathbf{r}$, since the message encoding is added to v_0 before compression (as shown in Lines 19 and 21 of the Algorithm 2). Even if one treated it as an MLWR sample, its rounding noise would be larger than that of the c_1 -based samples due to the coarser compression precision $d_v < d_u$. Nevertheless, to be conservative, we assume the same noise level as c_1 and incorporate this additional MLWR sample into our evaluation. Thus, in the ciphertext-based recovery setting, we assume the adversary has access to $k + 1$ MLWR samples about $\mathbf{r}$ (compared to k MLWR samples about $\mathbf{s}$ from the public key), and their noise level is determined by d_u . Consequently, ciphertext-based recovery is expected to be easier than public-key-based recovery for the same parameter set.

Cost model and known attacks. Our concrete security estimates are based on the Core-SVP model [7], a conservative cost model in which the time complexity of the BKZ algorithm with block size β is approximated by a single call to an SVP oracle in dimension β . For large β , sieving is the most efficient method for solving SVP in dimension β : the best known classical and quantum sieving complexities are $2^{0.292\beta}$ [9] and $2^{0.257\beta}$ [13], respectively.

In our evaluation, we consider the main known attack families against module-lattice schemes: primal attack [7, 4], BDD (hybrid) attacks [25, 3, 24], and dual attacks [12, 35]. We do not consider BKW attack [2], algebraic attack [8] and combinatorial attacks [36, 30] as they are far less competitive than primal and dual attacks in the WEAVER parameter regime. To evaluate the hardness of MLWR, one usually first transforms MLWR instances into standard LWE instances and then assesses the concrete computational complexity. Specifically, the deterministic rounding operation $\text{Compress}_q(\cdot, d)$ is modeled by introducing an additive, near-uniform rounding error e_{round} scaled back to $\mathbb{Z}_q$. Under this model, the decompressed representative satisfies:

$$\text{Decompress}_q(\text{Compress}_q(x, d), d) = x + e_{\text{round}} \pmod{q},$$

where e_{round} is modeled as a random variable uniformly distributed in the interval $[-q/2^{d+1}, q/2^{d+1}]$. The variance of this equivalent rounding noise is:

$$\sigma_{\text{round}}^2 \approx \frac{q^2}{12 \cdot 2^{2d}}.$$

By treating this rounding error as independent and identically distributed (i.i.d.) additive Gaussian-like noise with variance σ_{round}^2 , we can seamlessly input these concrete parameters (the module rank k , dimension n , modulus q , secret distribution parameter η , and the effective noise variance σ_{eff}^2) into state-of-the-art lattice estimators to compute the concrete Core-SVP security.

The study in [24] exploits the ring structure of $\mathbb{Z}_q[X]/(X^n + 1)$ to speed up BDD hybrid attacks [25, 3] and the dual attack in [12]. We note that the MATZOV dual attack [35] relies on several controversial assumptions [18]. The work of Carrier et al. [12] provides a refined analysis and experimental results that support cost estimates close to those claimed in [35], without relying on the contested assumptions. Following the approach of [20], we also consider it prudent to include the dual attacks from both [12, 35] in our cost estimates, notwithstanding the controversial assumptions underlying [35]. For the dual attack in [12], we adapted the evaluation code of [12] to accommodate our algorithm’s parameters—in particular, to handle the near-uniform distribution of error distribution in WEAVER. We further observed that the ring structure can be even more fully exploited within the dual attack framework of [12] compared to the analysis in [24], and that the asymmetry between the standard deviations of the secret dis-

tribution and error distribution provides additional advantage for accelerating the dual attack. We incorporated these observations into our evaluation code.

Table 5 details the individual attack costs for public-key-based recovery (recover $\mathbf{s}$) for each parameter set under the Core-SVP model obtained via the lattice estimator¹ [5] together with our adapted evaluation code of the latest dual attack [12], with the best cost (i.e., the minimum) bolded for each category. For ciphertext-based recovery (recover $\mathbf{r}$), the adversary has access to $k + 1$ MLWR samples according to our assumption rather than k samples from the public key. The corresponding attack costs for all parameter sets are listed in Table 6. Comparing Table 5 and Table 6, we observe that the costs for WEAVER-1024 and WEAVER-2048 are identical, showing that the additional MLWR sample from c_2 does not lead to a reduction in the estimated attack cost.

Table 7 provides the corresponding attack costs against Kyber under the same estimation methodology for comparison.

Table 5: Attack costs for public-key-based recovery (recover $\mathbf{s}$) against WEAVER’s underlying MLWR instance in the Core-SVP model.

Cost model	Primal	BDD	Dual [35]	Dual [12]
Weaver-640				
Classical	165.6	166.6	159.4	159.3
Quantum	145.7	146.7	141.6	150.8
Weaver-1024				
Classical	276.2	277.0	279.2	264.7
Quantum	243.1	243.9	248.6	237.9
Weaver-2048				
Classical	565.0	566.0	533.3	527.2
Quantum	497.3	498.3	473.5	480.2

Table 6: Attack costs for ciphertext-based recovery (recover $\mathbf{r}$) against WEAVER’s underlying MLWR instance in the Core-SVP model.

Cost model	Primal	BDD	Dual [35]	Dual [12]
Weaver-640				
Classical	160.6	161.5	153.6	153.6
Quantum	141.3	142.2	137.0	146.6
Weaver-1024				
Classical	276.2	277.0	279.2	264.7
Quantum	243.1	243.9	248.6	237.9
Weaver-2048				
Classical	565.0	566.0	533.3	527.2
Quantum	497.3	498.3	473.5	480.2

Interpretation of the reported security levels. The final security level reported in Table 4 is taken as the minimum cost across all considered attacks, i.e., the minimum among the public-key-based costs in Table 5 and the ciphertext-based costs in Table 6.

¹<https://github.com/malb/lattice-estimator>

Table 7: Attack costs against KYBER’s underlying MLWE instance in the Core-SVP model.

Cost model	Primal	BDD	Dual [35]	Dual [12]
Kyber-512				
Classical	118.6	119.4	115.5	118.5
Quantum	104.3	105.2	103.2	118.1
Kyber-768				
Classical	182.2	183.1	174.3	172.3
Quantum	160.4	161.2	155.7	160.7
Kyber-1024				
Classical	255.2	256.1	241.8	236.7
Quantum	224.6	225.5	215.4	213.0

The classical and quantum security figures in Table 4 should therefore be read as follows: they represent the estimated cost of the best known attacks against the underlying module-lattice problem instantiated by WEAVER, after accounting for the effective noise introduced jointly by deterministic rounding. In particular, these estimates support the claim that aggressive public key and ciphertext compression improves bandwidth efficiency without creating any currently known attack path that is asymptotically or concretely stronger than standard MLWE-based cryptanalysis.

Overall, our concrete-security assessment combines three ingredients: the standard hardness of MLWE at the uncompressed algebraic core, the rounding-based interpretation of the deployed compressed distribution, and the absence of known attacks that exploit compression beyond its effect on the effective noise level. Under this interpretation, the security levels reported for the proposed parameter sets remain comfortably above their intended targets.

Rationale for the 128-bit encapsulated secret at the 128-bit classical level. For the 128-bit classical security level, WEAVER encapsulates a 128-bit session key. This choice is intentional and closely follows the transition-oriented security requirements of the New Generation Commercial Cryptography (NGCC) algorithm call: the 128-bit public-key level is primarily designed to be used in conjunction with existing 128-bit symmetric primitives during the migration and transition period. In particular, a 128-bit encapsulated secret naturally matches the security strength, key sizes, and design paradigms of currently deployed 128-bit block ciphers (such as SM4) and hash-based key-derivation components.

We emphasize that this choice does not reduce the quantum security of the KEM below the required 80-bit level. A generic quantum exhaustive-search attack against a 128-bit encapsulated secret using Grover’s algorithm requires roughly 2^{64} oracle queries. However, these are strictly oracle-query counts; each Grover iteration must implement a non-trivial verification oracle. In the case of a KEM constructed via the Fujisaki-Okamoto (FO) transform [23, 27], this oracle must, at a minimum, check whether a candidate message re-generates the target ciphertext under the deterministic FO encryption procedure. Thus, the cost of one oracle call includes the quantum circuit cost of evaluating the corresponding encryption procedure and performing a coherent ciphertext comparison.

This is analogous to the standard interpretation of AES-128 key search in the NIST PQC security categories [37]. Although Grover’s algorithm reduces exhaustive search over a 128-bit AES key to about 2^{64} oracle queries, the AES oracle itself has non-negligible quantum circuit complexity. The standard cryptanalytic consensus is commonly interpreted as assigning at least 2^{16} elementary quantum gates to a single AES-128 oracle evaluation [21, 26], leading to an overall quantum gate complexity of approximately $2^{64} \cdot 2^{16} = 2^{80}$ for AES-128 key search.

The verification oracle for WEAVER is substantially more complex than a single AES-128 evaluation. Even ignoring the cost of the hash, XOF, and PRF components and focusing solely on the algebraic core, one deterministic re-encryption involves sampling from Centered Binomial Distributions (CBD), randomized lifting of the compressed public key, NTT-domain module arithmetic, multiple polynomial multiplications over R_q , coefficient-wise compression, and ciphertext comparison. For the 128-bit instance, the algebraic core performs a module-vector multiplication $\mathbf{A}^T \mathbf{r}$ (requiring k^2 polynomial multiplications) and an inner product $\mathbf{t}^T \mathbf{r}$ (requiring k polynomial multiplications) in the NTT domain, alongside forward and inverse NTTs, coefficient-wise modular arithmetic, and compression of all ciphertext components. Consequently, the quantum circuit implementing one WEAVER re-encryption oracle is conservatively no cheaper than an AES-128 oracle, and in practice, its gate count is expected to be significantly higher.

Therefore, even a purely generic Grover attack against the 128-bit encapsulated secret has a total quantum gate complexity that is at least comparable to, and realistically larger than, the 2^{80} reference cost associated with AES-128 key search. This generic-search argument is independent of, and weaker than, our concrete lattice-attack estimates in Table 4, which demonstrate that the best known quantum lattice attacks against the underlying module-lattice problem require substantially more than 2^{80} operations. Hence, the use of a 128-bit encapsulated secret at the 128-bit classical level is fully consistent with both the NGCC transition rationale and the required 80-bit quantum security lower bound.

4.3 Side-Channel Resistance and Implementation Security

Timing attacks Our reference implementation of WEAVER was evaluated against timing side-channel vulnerabilities using TIMECOP [38, 1]. Both the key generation and encapsulation processes exhibited a specific timing violation originating from the rejection sampling routine, which converts uniform random bytes into uniform random integers modulo q . In the key generation phase, the data triggering this leakage are derived from $seed_A$ (Lines 3–6 in Alg. 1), which is publicly disclosed as part of the public key. Similarly, the encapsulation process invokes rejection sampling during the randomized lifting via Inv_q (Line 3 in Alg. 2), which operates entirely on public randomized inputs and thus poses no security threat. Meanwhile, the decapsulation process demands rigorous engineering oversight. Because BCH decoding relies heavily on table lookups indexed by secret values, attackers could potentially exploit variations in execution time to extract sensitive information. Our preliminary evaluation indicates that achieving a constant-time decapsulation implementation introduces an approximate 10% computational overhead compared to the reference version. Notably, this analysis excludes timing leakage stemming from variable-time CPU instructions (e.g., KyberSlash [10]); cryptographic engineers must independently verify whether the underlying instruction set introduces such micro-architectural timing hazards.

Data-dependent Power/EM leakage We expect that any implementation of WEAVER lacking dedicated countermeasures will be vulnerable to power/EM based side-channel attacks. Implementing countermeasures, such as masking, requires rewriting multiple computation units (e.g., the NTT, polynomial multiplication, and message encoding) into shared versions. In a first-order masked implementation, every intermediate state x is split into two secret shares (x_1, x_2) such that $x = x_1 \oplus x_2$. For a general-purposed micro-controller (i.e. without a proper crypto-coprocessor), a first order masked implementation of ML-KEM could take around $5 \times$ executing time [22]. We expect the masking overhead WEAVER to be comparable to ML-KEM, yet users must customize the appropriate countermeasures according to their application requirements. Software masking schemes on general-purposed micro-controllers (e.g. ARM Cortex-M4) often failed to provide sufficient security, considering the measured leakage contains very limited random noise [17].

4.4 Other Security Properties

In addition to standard IND-CCA security, KEMs deployed in high-security network protocols often require stronger binding guarantees between the public key, the ciphertext, and the derived shared secret. Recent symbolic and computational analyses by Cremers, Dax, and Medinger [16] demonstrate that standard IND-CCA security does not inherently guarantee protocol-level binding properties. If a KEM lacks sufficient binding between the shared secret and the recipient’s public key, adversaries can exploit this gap to mount re-encapsulation or unknown-key-share (UKS) attacks on key-exchange protocols (such as those analyzed in [11, 39]), even if the underlying KEM is proven IND-CCA secure.

Key-to-public-key binding. To mitigate these protocol-level vulnerabilities, WEAVER incorporates the recipient’s public key directly into the Fujisaki–Okamoto (FO) transform derivation [23, 27]. Specifically, we hash the public key before deriving both the candidate shared secret and the deterministic encryption coins. During encapsulation (Algorithm 8), a random message m is sampled, and the joint state is derived via:

$$(K, r) = G(m \parallel H(pk)),$$

where $H(pk)$ acts as a public-key-binding commitment. The ciphertext is then generated as $ct = \text{PKE.Enc}(pk, m; r)$.

During decapsulation (Algorithm 9), the receiver reconstructs the candidate key and re-encryption coins using the same public key hash:

$$(K', r') = G(m' \parallel H(pk)),$$

and verifies the ciphertext consistency via $ct' = \text{PKE.Enc}(pk, m'; r')$. If the re-encryption check succeeds ($ct' = ct$), the candidate key K' is returned; otherwise, the algorithm falls back to the implicit rejection path.

The explicit inclusion of $H(pk)$ ensures that the derived key and the encryption coins are cryptographically bound to the public key. Even if an adversary attempts to replay or force the same internal message m under two distinct public keys $pk_1 \neq pk_2$, the derived outputs

$$G(m \parallel H(pk_1)) \quad \text{and} \quad G(m \parallel H(pk_2))$$

are computationally independent in the random oracle model, except with negligible probability of a hash collision. This provides key-to-public-key binding for honestly generated keys, which is an important building block for mitigating cross-recipient re-encapsulation and unknown-key-share attacks in higher-level authenticated key-exchange protocols [16].

Ciphertext-dependent implicit rejection. To prevent active side-channel attacks and Bleichenbacher-style vulnerabilities, WEAVER avoids explicit error symbols (such as $\perp$) and utilizes an implicit rejection mechanism. When ciphertext verification fails ($ct' \neq ct$), the decapsulation algorithm returns a pseudorandom fallback key:

$$K_{\text{rej}} = J(z, ct),$$

where z is a secret rejection seed stored in the KEM secret key. By hashing the received ciphertext ct into J , the rejection output is strictly ciphertext dependent. This ensures that malformed or modified ciphertexts do not map to a static recipient-wide rejection key, thereby preventing adversaries from collecting meaningful oracle feedback across multiple invalid queries.

Multi-target domain separation. The integration of $H(pk)$ into the random oracle G also provides robust domain separation in multi-target deployment scenarios. In a large-scale network where an adversary targets multiple public keys simultaneously, the adversary cannot amortize their cryptanalytic efforts across different targets. Since both the encryption coins r and the shared secret K depend on the recipient’s public key hash, each target public key defines an independent random-oracle domain. This prevents cross-recipient key-reuse and ciphertext-substitution attacks, ensuring that WEAVER remains highly resilient in multi-user environments.

5 Performance Evaluation and Implementation Analysis

This section presents a comprehensive evaluation of the proposed scheme’s performance across different software platforms. We analyze the computational complexity, storage requirements, and communication overhead, demonstrating the algorithm’s efficiency for both high-performance desktop processors and resource-constrained embedded devices.

5.1 Evaluation Methodology and Platforms

To ensure reproducibility and demonstrate the scheme’s versatility across deployment classes, we define the benchmarking environments and metrics as follows.

Metric definition. On x86-64, performance is reported as **median CPU cycles** measured via RDTSC with call overhead subtracted (`cpucycles_overhead()`); each entry is the median over five independent runs of $N_{\text{test}}=1,000$ timed iterations per run. On embedded targets, performance is likewise reported as median CPU cycles (median of $N_{\text{test}}=10,000$ runs); we additionally report maximum stack usage to assess suitability for memory-constrained devices.

- **High-performance platform (x86_64).** Single-socket KVM/QEMU guest reporting an **AMD EPYC 9655P** (96-core class, firmware clock **2.0 GHz**, 2 vCPUs, 128 KiB L1d/i per core, 1 MiB L2 per core); host ISA features include AVX2, BMI2, POPCNT, and AVX-512.
 - *OS:* Linux **6.1.0-49-amd64** (Debian 12)
 - *Compiler:* GCC **12.2.0**
 - *Flags:* `-std=c99 -O3 -fomit-frame-pointer -march=native -mtune=native -mavx2 -mbmi2 -mpopcnt -flto`
 - *RNG (benchmark path):* Linux `getrandom(2)` (`USE_GETRANDOM=1`), matching the ML-KEM reference; KAT paths retain the in-tree AES-256-CTR-DRBG.

All tables in §5.2 were collected on this platform.

- **Embedded platform (32-bit ARM).** STM32F407 Discovery Board (ARM Cortex-M4) @ 24 MHz, 192 KiB SRAM, 1 MiB Flash.
 - *Compiler:* `arm-none-eabi-gcc 14.3.rel1`
 - *Flags:* `-O3 -mthumb -mcpu=cortex-m4 -mfloat-abi=hard -mfpu=fpv4-sp-d16`
- **Embedded platform (32-bit RISC-V).** CM32M433R_START Board (RV32IMACF) @ 144 MHz, 144 KiB SRAM, 512 KiB Flash.
 - *Compiler:* `riscv64-unknown-elf-gcc 14.2.1`
 - *Flags:* `-O3 -march=rv32ima-fc -mabi=ilp32f`

5.2 Software Performance on x86 Processors

We evaluate the software performance of WEAVER on the x86_64 platform using both portable ISO C99 reference builds and AVX2-optimized implementations. To ensure a rigorous and fair comparison, all benchmarks are conducted using the same Keccak/SHAKE128 symmetric layer as the standard ML-KEM baseline (commit 4768bd3), isolating the computational characteristics of the underlying lattice and error-correction designs.

The AVX2 implementation vectorizes all core arithmetic operations, including polynomial arithmetic, parallel matrix sampling, centered binomial sampling (CBD), and bit-packing. For WEAVER-640, it enables a highly optimized AVX2 NTT at $n = 128$ and SHAKE128 $\times 4$ parallel matrix generation. For the higher-level instances (WEAVER-1024 and WEAVER-2048), we implement a dedicated $q = 7681$ AVX2 NTT alongside fused vector-multiply-accumulate operations (`polyvec.basemul_acc`).

5.2.1 Weaver: Reference vs. AVX2

Table 8 presents the self-comparison of WEAVER between the portable reference and AVX2-optimized builds. The AVX2 vectorization delivers substantial and consistent speedups across all security levels, yielding $1.6\times$ to $2.5\times$ performance gains. The most pronounced acceleration is achieved in the $q = 7681$ NTT kernels and parallel matrix expansion, demonstrating that the algebraic structures of WEAVER are highly amenable to SIMD parallelization.

Table 8: Weaver KEM performance: portable `ref/` vs. AVX2 (median CPU cycles, Keccak/SHAKE128-matrix config).

Scheme	Impl.	KeyGen	Encaps	Decaps
WEAVER-640	Ref (C99)	86,579	93,911	105,559
(128, 5, 3329)	AVX2	35,151	46,799	51,427
WEAVER-1024	Ref (C99)	125,475	149,759	172,379
(256, 4, 7681)	AVX2	57,303	71,265	83,615
WEAVER-2048	Ref (C99)	305,447	398,397	386,983
(512, 4, 7681)	AVX2	175,733	253,707	222,169

5.2.2 Comparison with ML-KEM and Rationale

Table 9 compares the end-to-end performance of WEAVER against ML-KEM. Rather than viewing these numbers as a direct drop-in comparison, they should be interpreted under the design philosophy of WEAVER, which deliberately trades a moderate increase in highly optimized AVX2 cycles for **superior security redundancy, robust correctness, and tighter bandwidth compression**.

No Algorithmic Bottleneck in Reference Parity. The most compelling evidence of WEAVER’s computational efficiency is found in the portable C99 reference comparison. At the matched $(n, k) = (256, 4)$ configuration, WEAVER-1024’s full encapsulation is practically on par with ML-KEM-1024 ($1.0\times$, 149,759 vs. 150,903 cycles), while its key generation is actually **16.4% faster** (125,475 vs. 150,201 cycles).

Furthermore, as shown in Table 10, when we strip away the common symmetric matrix-expansion overhead to isolate the core lattice and encoding arithmetic, WEAVER-1024’s reference

Table 9: End-to-end KEM performance: Weaver vs. ML-KEM, reference and AVX2 (median CPU cycles, Keccak/SHAKE128 on Weaver).

	Target	Scheme	Impl.	KeyGen	Encaps	Decaps
128-bit		ML-KEM-512	Ref	58,083	65,311	87,931
		ML-KEM-512	AVX2	10,841	11,595	12,713
		WEAVER-640	Ref	86,579	93,911	105,559
		WEAVER-640	AVX2	35,151	46,799	51,427
256-bit		ML-KEM-1024	Ref	150,201	150,903	188,759
		ML-KEM-1024	AVX2	23,503	24,179	26,727
		WEAVER-1024	Ref	125,475	149,759	172,379
		WEAVER-1024	AVX2	57,303	71,265	83,615

encapsulation is **17.2% faster** than ML-KEM-1024 (95,212 vs. 114,972 cycles). This reference-level parity and advantage clearly demonstrate that WEAVER’s mathematical design and error-correcting pipeline introduce **no intrinsic algorithmic bottlenecks**.

Table 10: Encapsulation with one matrix-XOF call removed (median cycles). Each “–Matrix XOF” entry is `kem_encaps` minus one `gen_matrixT` (Weaver) or `gen_a` (ML-KEM). Ratios compare Weaver to ML-KEM within the same block only.

Tier	Scheme	Full Encaps	–Matrix XOF	Weaver/ML-KEM [†]
Portable reference (C99)				
128-bit indicative	ML-KEM-512	65,311	56,290	0.91×
	WEAVER-640	93,911	51,298	
256-bit matched	ML-KEM-1024	150,903	114,972	0.83×
	WEAVER-1024	149,759	95,212	
AVX2 optimized				
128-bit indicative	ML-KEM-512	11,595	9,568	3.6×
	WEAVER-640	46,799	34,164	
256-bit matched	ML-KEM-1024	24,179	16,120	3.8×
	WEAVER-1024	71,265	61,568	

[†] Ratio = Weaver / ML-KEM in the “–Matrix XOF” column of the *same block*. 128-bit rows are *indicative* only ((n, k) differ); 256-bit rows are matched at $(n, k)=(256, 4)$. AVX2 stripped values use `gen_a/gen_matrixT` medians: 11,595–2,027, 46,799–12,635, 24,179–8,059, 71,265–9,697.

Source of the AVX2 Performance Gap. The performance gap observed on highly optimized AVX2 builds (e.g., $2.9\times$ on encapsulation at the 256-bit level) is a deliberate and expected consequence of our conservative parameter choices, rather than an implementation deficit:

1. **Conservative Noise and Modulus:** To achieve highly robust security margins, WEAVER-1024 employs a larger noise parameter ($\eta_1 = 7$) and a larger prime modulus ($q = 7681$). The larger noise requires generating $3.5\times$ more pseudorandom bytes per polynomial from the PRF, and the $q = 7681$ AVX2 NTT is naturally more complex than ML-KEM’s highly optimized $q = 3329$ assembly NTT.
2. **Negligible Overhead from Core Innovations:** Crucially, our core design innovations—namely, the Randomized Lifting (Inv_q) and the hierarchical BCH message encoding—are

extremely lightweight. As shown in our component-level profiling (Table 11), Inv_q and BCH encoding together consume only 5,380 cycles during Category-3 encapsulation, accounting for **about 7.5%** of the total encapsulation cost.

This decomposition confirms that WEAVER’s novel error-correction and randomized lifting frameworks are highly efficient and introduce virtually no computational bottleneck. The scheme successfully delivers a robust post-quantum security margin and a significantly reduced communication footprint (saving 9.2% bandwidth at Category 3) while maintaining highly competitive execution speeds well-suited for practical deployments.

Table 11: Encapsulation overhead breakdown: Weaver vs. ML-KEM (AVX2/Keccak, median cycles). Components marked $\dagger$ are absent in ML-KEM. The (128, 5) column compares WEAVER-640 against ML-KEM-512 at the same 128-bit target; dimensions differ.

Component	(128, 5) vs. ML-KEM-512			(256, 4) vs. ML-KEM-1024		
	ML-KEM	Weaver	Δ	ML-KEM	Weaver	Δ
<code>gen_matrix</code>	2,027	13,181	+11,154	8,059	9,983	+1,924
<code>poly_getnoise</code> (CBD)	2,520	4,522	+2,002	1,974	13,519	+11,545
NTT + basemul	309	10,084	+9,775	387	13,490	+13,103
Inv_q ($\dagger$)	—	5,121	+5,121†	—	4,731	+4,731†
BCH MsgEncode ($\dagger$)	25	363	+338†	25	649	+624†
Compress + pack	525	2,286	+1,761	525	4,756	+4,231
randombytes + hash	3,350	6,002	+2,652	5,500	15,311	+9,811
Total (KEM Encaps)	11,595	46,799	+35,204	24,179	71,265	+47,086

† Direct cost of randomized lifting Inv_q (tight MLWR reduction) and BCH coding (aggressive compression). Weaver Inv_q /BCH/`gen_matrix`/CBD/compress rows are segment medians from `bin/profile_encap_512` and `bin/profile_encap_1024` (`tools/profile_encap_decaps.c`); ML-KEM rows from the Kyber AVX2 harness. Weaver `poly_getnoise` sums k polys ($k=4$ at (256, 4)); ML-KEM segments use the Kyber harness definition (typically fewer calls or a partial sum—use per-poly microbenchmarks above for η -normalized comparison).

5.3 Performance on Embedded Systems

In this section, we further investigate the performance of our reference implementation (`ref/`) on two embedded micro-processors (STM32F407 for ARM and CM32M433 for RISC-V), demonstrating its versatility across a variety of resource-constrained devices.

5.3.1 Experimental Setup

Within this section, we choose one ARM Cortex-M4 core (STM32F407 Discovery Board) and one RISC-V RV32 core (CM32M433R START Board) as representatives for commonly used embedded processors. Although both have rather limited RAM (192KB & 144KB), even the reference implementation of WEAVER can naturally fit in such memory-constrained devices. For the STM32F407, cycle counts are calculated using the hardware DWT (Data Watchpoint and Trace) unit embedded inside the ARM Cortex-M4 core. For the CM32M433R, cycle counts are read from the mcycle Control and Status Register (CSR). Each entry in the following tables is the median of $N_{\text{test}}=10,000$ iterations. The initial seed of PRNG is set to a 48-byte constant and the PRNG is not reset during $N_{\text{test}}=10,000$ iterations. Stack memory cost is estimated using the same technique as `pqm4` (i.e. writing a random canary to the entire available stack space, then checking the memory space that have been overwritten) [28].

Table 12: Performance and Stack Usage on ARM Cortex-M4 (STM32F407 @ 24MHz).

Scheme	(n, k)	Sec.	KeyGen	Encaps	Decaps	Stack Usage (B)
ML-KEM-512	(256, 2)	128-bit	565,496	669,905	849,134	6,152/8,784/9,560
WEAVER-640	(128, 5)	128-bit	1,074,402	1,287,731	1,474,420	10,168/12,032/12,840
ML-KEM-768	(256, 3)	192-bit	937,126	1,081,400	1,318,115	10,248/13,384/14,480
ML-KEM-1024	(256, 4)	256-bit	1,455,332	1,629,573	1,928,045	15,128/18,776/20,352
WEAVER-1024	(256, 4)	256-bit	1,745,907	2,121,443	2,497,196	13,496/16,624/18,136
WEAVER-2048	(512, 4)	512-bit	3,633,522	4,531,632	5,406,820	26,288/32,584/35,920

Table 13: Performance and Stack Usage on RV32IMACF (CM32M433 @ 144 MHz).

Scheme	(n, k)	Sec.	KeyGen	Encaps	Decaps	Stack Usage (B)
ML-KEM-512	(256, 2)	128-bit	1,618,343	1,792,334	2,113,011	6,160/8,784/9,552
WEAVER-640	(128, 5)	128-bit	3,445,814	3,926,785	4,292,227	10,224/12,096/12,896
ML-KEM-768	(256, 3)	192-bit	2,713,398	2,983,468	3,459,099	10,240/13,392/14,480
ML-KEM-1024	(256, 4)	256-bit	4,282,388	4,619,461	5,176,147	15,072/18,720/20,288
WEAVER-1024	(256, 4)	256-bit	5,277,447	6,046,591	6,829,381	13,504/16,624/18,128
WEAVER-2048	(512, 4)	512-bit	10,581,253	12,632,832	14,508,493	26,304/32,592/35,952

5.3.2 Benchmarking Results

On STM32F407. Table 12 details the cycle counts and stack usage on STM32F407 (ARM Cortex-M4). Following the guidance in pqm4 [28], we set the MCU frequency to 24 MHz such that the CPU has zero wait states when fetching instructions. Without explicit assembly optimization, WEAVER-640 takes $1.9\times$ execution time and $1.3\times$ stack memory (vs. ML-KEM-512). However, for higher security levels (e.g. 256-bit), WEAVER-1024 takes only $1.3\times$ execution time and $0.9\times$ stack memory (vs. ML-KEM-1024). For 512-bit security level, the stack usage cost of WEAVER-2048 (around 30KB) still fits in many resource-constrained devices.

On CM32M433. Table 13 reports the performance of our reference C-based implementation on the CM32M433 (RV32IMACFP). Unlike STM32F407, all benchmarks are conducted at the maximum operating frequency of 144 MHz, where the processor may incur additional latency due to flash-memory instruction fetching. Compared with ML-KEM, WEAVER shows a performance overhead of approximately $2.1\times$ – $2.5\times$ across all parameter sets, while requiring only a moderate increase in stack memory. Interestingly, although WEAVER incurs a noticeable runtime overhead, its stack usage remains comparable to that of ML-KEM. In particular, WEAVER-1024 consistently requires less stack memory than ML-KEM-1024 across key generation, encapsulation, and decapsulation.

5.4 Optimization Techniques

To achieve the performance in §5.2, we apply platform-specific optimizations on top of the portable `ref/` baseline (`avx2_weaver/` on x86_64; ISO C on embedded targets).

5.4.1 Vectorization with AVX2 (x86_64)

The AVX2 tree keeps the Keccak/SHAKE symmetric layer aligned with ML-KEM (SHAKE128 matrix XOF, SHAKE256 PRFs) and vectorizes the polynomial layer:

- **Modulus-specific NTT kernels.** At $q=3329$ we reuse a Kyber-style AVX2 NTT/basemul stack ($n=128$ for WEAVING-640, $n=256$ where applicable). At $q=7681$ we ship a dedicated AVX2 NTT for WEAVING-1024 ($n=256$) and WEAVING-2048 ($n=512$), operating in *standard coefficient order* (no `nttunpack` on the hot path). Butterflies use `vpmullw/vpmulhw` Montgomery multiplication with staged (lazy) reduction between layers.
- **Parallel matrix expansion.** `gen_matrix` / `gen_matrixT` are implemented with $\text{SHAKE128} \times 4$: four XOF streams are absorbed and squeezed in parallel, and rejection sampling fills matrix entries concurrently. This is the largest single kernel after NTT on $q=7681$ modes.
- **Fused module arithmetic.** `polyvec_basemul_acc` fuses the inner product over module rank k , reducing $\text{NTT} \rightarrow \text{basemul} \rightarrow \text{accumulate}$ traffic for both key generation and encapsulation.
- **AVX2 compression and (de)serialization.** WEAVING-640 uses AVX2 helpers for 9-bit public-key compression; ciphertext packing follows the scheme-specific (d_u, d_v) bit widths from Table 2.
- **AVX2 centered binomial sampling (CBD).** For $\eta \in \{2, 3, 7\}$ we vectorize CBD (Kyber-style $\eta=2/3$; SWAR 32-bit popcount for $\eta=7$). For $\eta=9$ (WEAVING-2048) the reference bit-extraction path is retained: end-to-end CBD cost is still dominated by SHAKE256 PRF expansion, so the remaining gap vs. ML-KEM is primarily parameterization ($\eta_1=9$ vs. 2), not missing SIMD in the sampler core.
- **Build flags.** Release measurements use `-O3 -march=native -mavx2 -mbmi2 -mpopcnt -flto`; correctness is checked by byte-identical replay of `ref/` vectors (`compare_ref_avx.*`).

We do *not* count ML-KEM-unique FO artifacts; WEAVING-specific costs (Inv_q lifting, BCH message coding) remain scalar in the current AVX2 build and are analyzed separately in Table 11.

5.5 Transmission and Storage Overhead

Minimizing communication bandwidth and storage overhead is a primary design goal for practical post-quantum key encapsulation mechanisms. Table 14 presents the concrete serialized sizes of the public key (pk), ciphertext (ct), and the IND-CCA-secure secret key (sk) for the three proposed parameter sets of WEAVING.

Table 14: Data Sizes and Transmission Overhead of WEAVING Parameter Sets

Instance	$ pk $ (Bytes)	$ ct $ (Bytes)	Total $pk + ct$ (Bytes)	$ sk $ (Bytes)
WEAVING-640	752	816	1568	1776
WEAVING-1024	1312	1536	2848	3072
WEAVING-2048	2880	3392	6272	6400

At the Category-1 level, WEAVING-640 achieves a combined public-key-plus-ciphertext footprint of exactly 1568 bytes, which matches the transmission size of ML-KEM-512 ($800 + 768 = 1568$ bytes) byte-for-byte. However, as detailed in Section 4.4, WEAVING-640 delivers this matching wire size while providing a significantly higher estimated Core-SVP security margin and

robust multi-layer error-correcting redundancy. For higher security categories, WEAVER-1024 reduces the combined footprint by approximately 9.2% compared to ML-KEM-1024 (2848 bytes vs. 3136 bytes), demonstrating the scalability of our aggressive compression and hierarchical coding strategy.

In terms of network protocol compatibility, particularly for UDP-based protocols such as IKEv2/IPsec [29], a combined footprint of 1568 bytes is extremely close to the standard Ethernet Maximum Transmission Unit (MTU) of 1500 bytes (excluding IP/UDP headers). While a single handshake payload containing both the public key and ciphertext will typically trigger IP fragmentation or require fragmentation handling at the protocol layer, WEAVER-640 preserves the exact fragmentation profile of ML-KEM-512, ensuring a seamless drop-in transition without introducing additional packet-loss risks.

Secret Key Layout and Storage. To support constant-time decapsulation and robust key-to-public-key binding, the IND-CCA-secure secret key sk_{KEM} is structured to cache critical intermediate values. This layout is defined consistently across our portable reference and AVX2-optimized implementations. As illustrated in Table 15, the total secret key size $WEAVER_SECRETKEYBYTES$ is the sum of the IND-CPA secret vector $\mathbf{s}$, the embedded public key pk , the public key hash $H(pk)$, and the implicit rejection seed z .

Table 15: Internal Layout and Field Sizes of the IND-CCA Secret Key (Bytes)

Field	Description	Weaver-640	Weaver-1024	Weaver-2048
sk_{PKE}	Packed secret vector $\mathbf{s}$ (uncompressed)	960	1664	3328
pk	Embedded public key copy	752	1312	2880
$H(pk)$	Public key hash commitment	32	64	128
z	Implicit rejection seed	32	32	64
Total	$WEAVER_SECRETKEYBYTES$	1776	3072	6400

The IND-CPA secret vector $\mathbf{s} \in R_q^k$ is packed without compression to prevent any leakage or degradation of the secret coefficients, requiring $k \times (n \times \lceil \log_2 q \rceil / 8)$ bytes, which translates to 960 bytes for WEAVER-640 (using $q = 3329$, 12 bits per coefficient), and 1664 and 3328 bytes for the higher levels (using $q = 7681$, 13 bits per coefficient). Caching the public key pk and its hash $H(pk)$ within sk_{KEM} avoids the computational overhead of re-serialization and re-hashing during the decapsulation re-encryption check. This design choice slightly increases the local storage footprint compared to ML-KEM (e.g., 1776 bytes for WEAVER-640 vs. 1632 bytes for ML-KEM-512) but guarantees optimal constant-time decapsulation performance on resource-constrained platforms.

6 Features Statement

6.1 Innovation: No-Compromise Security and Efficiency

The primary innovation of WEAVER lies in its **High-Density Information Packing via Parameter-Specific Error Correction**, which delivers robust security redundancy and rigorous correctness guarantees without sacrificing computational efficiency or transmission size.

Traditional lattice-based KEMs typically avoid explicit error-correcting codes to keep the implementation simple, relying instead on conservative scalar rounding that often leads to larger public keys or ciphertexts to maintain a negligible decryption failure probability (DFR). WEAVER breaks this paradigm by combining aggressive compression with a tailored, hierarchical error-correction pipeline across all parameter sets: a higher-layer BCH code and a lower-layer BCH-plus-4-way-repetition code. This two-layer design is instantiated with parameter-specific

BCH codes for WEAV-640, WEAV-1024, and WEAV-2048. This innovation directly translates to:

- **Aggressive Compression with High Noise Tolerance:** By leveraging parameter-specific error correction, WEAV can tolerate a significantly higher noise threshold during the LWE decryption phase. This allows the Compress operation to drop more bits, enabling tighter serialization of the public key and ciphertext while maintaining an exceptionally steep error-decay curve.
- **Byte-for-Byte Bandwidth Parity:** At the Category-1 level, WEAV-640 achieves a combined public-key-plus-ciphertext footprint of exactly 1568 bytes (752 bytes for pk and 816 bytes for ct), matching the transmission size of ML-KEM-512 byte-for-byte. This proves that our advanced coding layer does not impose any packaging penalty on the wire.
- **No Intrinsic Algorithmic Bottleneck:** Despite incorporating sophisticated BCH and randomized lifting (Inv_q) mechanisms, WEAV introduces no computational bottlenecks. As demonstrated in Section 5, our portable reference implementation achieves strict performance parity with ML-KEM (with WEAV-1024’s reference encapsulation matching ML-KEM-1024, and its key generation running 16.4% faster), while the core coding and lifting operations consume less than 7.6% of the total execution budget.

6.2 Compatibility and Real-World Applicability

The compact and balanced design of WEAV makes it exceptionally well-suited for real-world network protocols and legacy IT infrastructures.

Seamless Network Integration. A major pain point in migrating to Post-Quantum Cryptography (PQC) is the packet fragmentation issue in UDP-based protocols like IKEv2/IPsec and QUIC, where large public keys and ciphertexts exceed the standard Ethernet Maximum Transmission Unit (MTU) of 1500 bytes. Because WEAV-640’s combined footprint (1568 bytes) is identical to that of ML-KEM-512, it preserves the exact fragmentation profile and network behavior of the standard baseline. This allows WEAV to act as a seamless, drop-in replacement in existing network security architectures without requiring any modifications to MTU handling, fragmentation mitigation strategies, or protocol-layer buffering.

Robust Protocol-Level Security. By deploying WEAV, network protocols gain substantial security advantages. The integration of the public key hash $H(pk)$ into the Fujisaki–Okamoto derivation provides key-to-public-key binding, mitigating re-encapsulation and unknown-key-share (UKS) attacks at the protocol level. Furthermore, the high noise tolerance provided by our BCH-Repetition pipeline ensures that the physical decryption failure probability is kept cryptographically negligible, eliminating any potential multi-target or active side-channel exploits targeting decryption failures, all while maintaining competitive execution speeds on both x86 processors and resource-constrained embedded devices.

References

- [1] ebacs: Ecrypt benchmarking of cryptographic systems. <https://bench.cr.yp.to/supercop.html>. Accessed: 2026-06-17.
- [2] Martin R. Albrecht, Carlos Cid, Jean-Charles Faugère, Robert Fitzpatrick, and Ludovic Perret. On the complexity of the BKW algorithm on LWE. *Des. Codes Cryptogr.*, 74(2):325–354, 2015.
- [3] Martin R. Albrecht, Benjamin R. Curtis, and Thomas Wunderer. Exploring trade-offs in batch bounded distance decoding. In Kenneth G. Paterson and Douglas Stebila, editors, *Selected Areas in Cryptography - SAC 2019 - 26th International Conference, Waterloo, ON, Canada, August 12-16, 2019, Revised Selected Papers*, volume 11959 of *Lecture Notes in Computer Science*, pages 467–491. Springer, 2019.
- [4] Martin R. Albrecht, Florian Göpfert, Fernando Virdia, and Thomas Wunderer. Revisiting the expected cost of solving usvp and applications to LWE. In Tsuyoshi Takagi and Thomas Peyrin, editors, *Advances in Cryptology - ASIACRYPT 2017 - 23rd International Conference on the Theory and Applications of Cryptology and Information Security, Hong Kong, China, December 3-7, 2017, Proceedings, Part I*, Lecture Notes in Computer Science, pages 297–322. Springer, 2017.
- [5] Martin R. Albrecht, Rachel Player, and Sam Scott. On the concrete hardness of learning with errors. *Cryptology ePrint Archive*, 2015.
- [6] Erdem Alkim, Léo Ducas, Thomas Pöppelmann, and Peter Schwabe. Newhope without reconciliation. *Cryptology ePrint Archive*, 2016.
- [7] Erdem Alkim, Léo Ducas, Thomas Pöppelmann, and Peter Schwabe. Post-quantum key exchange - A new hope. In Thorsten Holz and Stefan Savage, editors, *25th USENIX Security Symposium, USENIX Security 16, Austin, TX, USA, August 10-12, 2016*, pages 327–343. USENIX Association, 2016.
- [8] Sanjeev Arora and Rong Ge. New algorithms for learning in presence of errors. In Luca Aceto, Monika Henzinger, and Jiri Sgall, editors, *Automata, Languages and Programming - 38th International Colloquium, ICALP 2011, Zurich, Switzerland, July 4-8, 2011, Proceedings, Part I*, volume 6755 of *Lecture Notes in Computer Science*, pages 403–415. Springer, 2011.
- [9] Anja Becker, Léo Ducas, Nicolas Gama, and Thijs Laarhoven. New directions in nearest neighbor searching with applications to lattice sieving. In Robert Krauthgamer, editor, *Proceedings of the Twenty-Seventh Annual ACM-SIAM Symposium on Discrete Algorithms, SODA 2016, Arlington, VA, USA, January 10-12, 2016*, pages 10–24. SIAM, 2016.
- [10] Daniel J. Bernstein, Karthikeyan Bhargavan, Shivam Bhasin, Anupam Chattopadhyay, Tee Kiah Chia, Matthias J. Kannwischer, Franziskus Kiefer, Thales B. Paiva, Prasanna Ravi, and Goutam Tamvada. Kyberslash: Exploiting secret-dependent division timings in kyber implementations. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2025(2):209–234, Mar. 2025.
- [11] Joppe Bos, Léo Ducas, Eike Kiltz, Tancrede Lepoint, Vadim Lyubashevsky, John M. Schanck, Peter Schwabe, Gregor Seiler, and Damien Stehlé. Crystals-kyber: a cca-secure module-lattice-based kem. In *2018 IEEE European symposium on security and privacy (EuroS&P)*, pages 353–367. IEEE, 2018.

- [12] Kévin Carrier, Charles Meyer-Hilfiger, Yixin Shen, and Jean-Pierre Tillich. Assessing the impact of a variant of matzov’s dual attack on kyber. In Yael Tauman Kalai and Seny F. Kamara, editors, *Advances in Cryptology - CRYPTO 2025 - 45th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 17-21, 2025, Proceedings, Part I*, volume 16000 of *Lecture Notes in Computer Science*, pages 444–476. Springer, 2025.
- [13] André Chailloux and Johanna Loyer. Lattice sieving via quantum random walks. In Mehdi Tibouchi and Huaxiong Wang, editors, *Advances in Cryptology - ASIACRYPT 2021 - 27th International Conference on the Theory and Application of Cryptology and Information Security, Singapore, December 6-10, 2021, Proceedings, Part IV*, volume 13093 of *Lecture Notes in Computer Science*, pages 63–91. Springer, 2021.
- [14] Long Chen, Zhenfeng Zhang, and Zhenfei Zhang. On the hardness of the computational ring-LWR problem and its applications. *Cryptology ePrint Archive*, Paper 2018/536, 2018.
- [15] Jung Hee Cheon, Hyeongmin Choe, Dongyeon Hong, and MinJune Yi. Smaug: Pushing lattice-based key encapsulation mechanisms to the limits. In *International Conference on Selected Areas in Cryptography*, pages 127–146. Springer, 2023.
- [16] Cas Cremers, Alexander Dax, and Niklas Medinger. Keeping up with the kems: Stronger security notions for kems and automated analysis of kem-based protocols. page 1046–1060, 2024.
- [17] Elena Dubrova, Kalle Ngo, Joel Gärtner, and Ruize Wang. Breaking a fifth-order masked implementation of crystals-kyber by copy-paste. In Masayuki Fukumitsu and Shingo Hasegawa, editors, *Proceedings of the 10th ACM Asia Public-Key Cryptography Workshop, APKC 2023, Melbourne, VIC, Australia, July 10-14, 2023*, pages 10–20. ACM, 2023.
- [18] Léo Ducas and Ludo N. Pulles. Does the dual-sieve attack on learning with errors even work? In Helena Handschuh and Anna Lysyanskaya, editors, *Advances in Cryptology - CRYPTO 2023 - 43rd Annual International Cryptology Conference, CRYPTO 2023, Santa Barbara, CA, USA, August 20-24, 2023, Proceedings, Part III*, volume 14083 of *Lecture Notes in Computer Science*, pages 37–69. Springer, 2023.
- [19] Jan-Pieter D’Anvers, Angshuman Karmakar, Sujoy Sinha Roy, and Frederik Vercauteren. Saber: Module-lwr based key exchange, cpa-secure encryption and cca-secure kem. In *International Conference on Cryptology in Africa*, pages 282–305. Springer, 2018.
- [20] Lewis Glabush, Patrick Longa, Michael Naehrig, Chris Peikert, Douglas Stebila, and Fernando Virdia. Frodokem: A cca-secure learning with errors key encapsulation mechanism. *IACR Commun. Cryptol.*, 2(3):25, 2025.
- [21] Markus Grassl, Brandon Langenberg, Martin Roetteler, and Rainer Steinwandt. Applying Grover’s algorithm to AES: Quantum resource estimates. In *Post-Quantum Cryptography - 7th International Workshop, PQCrypto 2016*, pages 29–43. Springer, 2016.
- [22] Daniel Heinz, Matthias J. Kannwischer, Georg Land, Thomas Pöppelmann, Peter Schwabe, and Amber Sprenkels. First-order masked kyber on ARM cortex-m4. *IACR Cryptol. ePrint Arch.*, 2022:58, 2022.
- [23] Dennis Hofheinz, Kathrin Hövelmanns, and Eike Kiltz. A modular analysis of the fujisaki-okamoto transformation. In *Theory of Cryptography Conference*, pages 341–371. Springer, 2017.
- [24] Jianhua Hou, Haodong Jiang, and Tabitha Ogilvie. Careful with the ring! concrete hardness gaps between lwe and mlwe. *To appear at CRYPTO 2026*, 2026.

- [25] Nick Howgrave-Graham. A hybrid lattice-reduction and meet-in-the-middle attack against NTRU. In Alfred Menezes, editor, *Advances in Cryptology - CRYPTO 2007, 27th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 19-23, 2007, Proceedings*, volume 4622 of *Lecture Notes in Computer Science*, pages 150–169. Springer, 2007.
- [26] Samuel Jaques, Michael Naehrig, Martin Roetteler, and Fernando Virdia. Implementing Grover oracles for quantum key search on AES and LowMC. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 280–310. Springer, 2020.
- [27] Haodong Jiang, Zhenfeng Zhang, Long Chen, Hong Wang, and Zhi Ma. Ind-cca-secure key encapsulation mechanism in the quantum random oracle model, revisited. In *Annual International Cryptology Conference*, pages 96–125. Springer, 2018.
- [28] Matthias J. Kannwischer, Joost Rijneveld, Peter Schwabe, and Ko Stoffelen. pqm4: Testing and benchmarking NIST PQC on ARM cortex-m4. *IACR Cryptol. ePrint Arch.*, 2019:844, 2019.
- [29] Charlie Kaufman, Paul Hoffman, Yoav Nir, Pasi Eronen, and Tero Kivinen. Internet key exchange protocol version 2 (ikev2). RFC 7296, Internet Engineering Task Force (IETF), October 2014.
- [30] Eunmin Lee, Joohee Lee, and Yuntao Wang. Improved meet-lwe attack via ternary trees. *IACR Cryptol. ePrint Arch.*, 2024:824, 2024.
- [31] Shu Lin, Daniel J Costello, et al. *Error control coding*, volume 2. Prentice hall Scarborough, 2001.
- [32] Feng-Hao Liu and Zhedong Wang. Rounding in the rings. In *Annual International Cryptology Conference*, pages 296–326. Springer, 2020.
- [33] Xianhui Lu, Yamin Liu, Zhenfei Zhang, Dingding Jia, Haiyang Xue, Jingnan He, Bao Li, and Kunpeng Wang. LAC: Practical ring-LWE based public-key encryption with byte-level modulus. Cryptology ePrint Archive, Paper 2018/1009, 2018.
- [34] Florence Jessie MacWilliams and Neil James Alexander Sloane. *The theory of error-correcting codes*, volume 16. Elsevier, 1977.
- [35] MATZOV. Report on the security of lwe: Improved dual lattice attack. 2022.
- [36] Alexander May. How to meet ternary LWE keys. In Tal Malkin and Chris Peikert, editors, *Advances in Cryptology - CRYPTO 2021 - 41st Annual International Cryptology Conference, CRYPTO 2021, Virtual Event, August 16-20, 2021, Proceedings, Part II*, volume 12826 of *Lecture Notes in Computer Science*, pages 701–731. Springer, 2021.
- [37] National Institute of Standards and Technology. Submission requirements and evaluation criteria for the post-quantum cryptography standardization process. NIST Post-Quantum Cryptography Project, 2016.
- [38] Moritz Neikes. Automated dynamic analysis for timing side-channels. <https://www.post-apocalyptic-crypto.org/timecop/>. Accessed: 2026-06-17.
- [39] Chris Peikert. Lattice cryptography for the internet. In *International workshop on post-quantum cryptography*, pages 197–219. Springer, 2014.

A From MLWE to Shifted-MLWR

We now justify the shifted-MLWR assumption used in the ciphertext part by relating it to the standard LWE-to-LWR proof framework. The point is not that shifted-MLWR is a black-box consequence of ordinary MLWR: given only a rounded value $\text{Compress}_q(x, d)$, one cannot in general compute $\text{Compress}_q(x + \Delta, d)$. Instead, we use the fact that the LWE-to-LWR proof of Liu–Wang [32] is stable under replacing the final rounding map by a publicly shifted rounding map. This is because the statistical steps in their proof establish closeness before rounding, and the public shift followed by rounding is a deterministic post-processing.

A.1 Definitions and known tools from Liu–Wang.

We recall the two ingredients from [32] that will be used.

Definition A.1 (Lossy sampler over ring, Definition 5.13 in [32]). *Let n, ℓ, f, p, q, k be positive integers, $R = \mathcal{O}_K$ be the ring of integers of a number field K of degree n , and ϕ be a distribution over $K_{\mathbb{R}}$. The algorithm $\text{Lossy}(1^n, 1^\ell, 1^f, 1^k, q, \phi)$ samples $\mathbf{D} \leftarrow \$(R_q)^{\ell \times f}$, $\mathbf{C} \leftarrow \$(R_q)^{f \times k}$, $\mathbf{F} \leftarrow \phi^{\ell \times k}$, and outputs $\tilde{\mathbf{A}} = \mathbf{DC} + \mathbf{F}$. We omit f when $f = 1$.*

Lemma A.2 (Algebraic leftover hash lemma (ALHL), Theorem 5.7 in [32]). *Let $R = \mathcal{O}_K$ be the ring of integers of a number field of degree n , and let q be an integer coprime to $[\mathcal{O}_K : \mathbb{Z}[\alpha]]$. For any distribution $\mathcal{X}$ over $(R_q^\vee)^k$, let $(\mathbf{A}, \mathbf{s}) \leftarrow \$(R_q)^k \times \mathcal{X}$, and $u \leftarrow \$(R_q^\vee)$ independent. Then*

$$\text{SD}((\mathbf{A}, \langle \mathbf{A}, \mathbf{s} \rangle), (\mathbf{A}, u)) \leq \frac{1}{2} \sqrt{\sum_{\substack{\mathfrak{q} \neq (1) \\ \mathfrak{q} | qR}} N(\mathfrak{q}) \cdot \text{Col}(\mathcal{X} \bmod \mathfrak{q}R^\vee)},$$

where $\text{Col}(\cdot)$ denotes collision probability. In particular, if for every ideal factor $\mathfrak{q} \mid qR$, $H_\infty(\mathbf{s} \bmod \mathfrak{q}R^\vee) \geq \tau$, then the statistical distance is bounded by $\varepsilon_{\text{lh}}(\tau)$, where ε_{lh} is as specified in Corollaries 5.9–5.11 of [32].

We will also use the following entropy preservation fact, which is a corollary of the Liu–Wang proof (specifically, Lemma 5.21 and surrounding text): in the lossy hybrids, conditioned on the lossy matrix and the rounded values, the secret retains enough smooth min-entropy modulo every relevant ideal factor, up to a leakage of $(n + \lambda) \log q$ bits. The bound is uniform over all public shifts, as shown in Step 2 of the proof below.

A.2 Product shifted rounding map.

Let $\mathbf{d} = (d_1, \dots, d_\ell)$ be a compression profile. For a public offset $\Delta = (\Delta_1, \dots, \Delta_\ell) \in \mathcal{R}_q^\ell$, define the shifted product rounding map

$$\mathcal{R}_{\mathbf{d}, \Delta} : \mathcal{R}_q^\ell \longrightarrow \prod_{i=1}^{\ell} (\mathbb{Z}_{2^{d_i}})^n$$

by

$$\mathcal{R}_{\mathbf{d}, \Delta}(\mathbf{x}) = \text{Compress}_{q, \mathbf{d}}(\mathbf{x} + \Delta) = (\text{Compress}_q(x_1 + \Delta_1, d_1), \dots, \text{Compress}_q(x_\ell + \Delta_\ell, d_\ell)).$$

When $\Delta = \mathbf{0}$, we write $\mathcal{R}_{\mathbf{d}, 0}(\mathbf{x}) = \text{Compress}_{q, \mathbf{d}}(\mathbf{x})$ for the ordinary product rounding map.

If rounding is defined with respect to a basis B in the sense of Liu–Wang, then each coordinate map $\text{Compress}_q(\cdot, d_i)$ should be interpreted as the basis-dependent coefficient rounding map $\lfloor \cdot \rfloor_{B, 2^{d_i}}$. In the implementation considered here, B is the coefficient basis used by the compression routine.

Definition A.3 (Shifted-MLWR). Let $k, \ell \in \mathbb{N}$, let χ_s be a secret distribution over $\mathcal{R}_q$, let $\mathbf{d}$ be a compression profile, and let $\Delta \in \mathcal{R}_q^\ell$ be a fixed public offset. The decisional $\text{Shifted-MLWR}_{k,\ell,\chi_s,\mathbf{d},\Delta}$ problem is to distinguish between the following two distributions:

- **Real:** sample $\mathbf{A} \leftarrow \$\mathcal{R}_q^{\ell \times k}$ and $\mathbf{s} \leftarrow \chi_s^k$, and output $(\mathbf{A}, \text{Compress}_{\mathbf{q}\mathbf{d}}(\mathbf{A}\mathbf{s} + \Delta))$.
- **Random:** sample $\mathbf{A} \leftarrow \$\mathcal{R}_q^{\ell \times k}$ and $\mathbf{u} \leftarrow \$\mathcal{R}_q^\ell$, and output $(\mathbf{A}, \text{Compress}_{\mathbf{q}\mathbf{d}}(\mathbf{u} + \Delta))$.

For any adversary $\mathcal{D}$, its distinguishing advantage is denoted by $\text{Adv}_{k,\ell,\chi_s,\mathbf{d},\Delta}^{\text{Shifted-MLWR}}(\mathcal{D})$.

Remark A.4 (Random branch equals ordinary MLWR random branch). By Lemma A.6 below, since $\mathbf{u} \leftarrow \$\mathcal{R}_q^\ell$ is uniform and Δ is a fixed public offset, $\mathbf{u} + \Delta$ is also uniform over $\mathcal{R}_q^\ell$. Therefore the random distribution of $\text{Shifted-MLWR}_{k,\ell,\chi_s,\mathbf{d},\Delta}$ is identical to that of ordinary $\text{MLWR}_{k,\ell,\chi_s,\mathbf{d}}$. The shift Δ affects only the real distribution, where it is absorbed into the inner product before compression.

Lemma A.5 (Post-processing under a public shift). Let X and Y be two distributions over a common domain, and let F be any deterministic function. Then

$$\text{SD}(F(X), F(Y)) \leq \text{SD}(X, Y).$$

In particular, for every fixed public shift Δ , applying $F_\Delta(\text{st}, \mathbf{z}) = (\text{st}, \mathcal{R}_{\mathbf{d},\Delta}(\mathbf{z}))$ to two statistically close distributions cannot increase their statistical distance.

Proof. This is the standard data-processing inequality for statistical distance. For completeness, for every event E in the range of F , the preimage $F^{-1}(E)$ is an event in the domain. Hence

$$|\Pr[F(X) \in E] - \Pr[F(Y) \in E]| = |\Pr[X \in F^{-1}(E)] - \Pr[Y \in F^{-1}(E)]| \leq \text{SD}(X, Y).$$

Taking the maximum over all events E proves the claim. $\square$

Lemma A.6 (Uniformity under public translation). Let $\mathbf{u} \leftarrow \$\mathcal{R}_q^\ell$ be uniform and let $\Delta \in \mathcal{R}_q^\ell$ be fixed. Then $\mathbf{u} + \Delta$ is also uniform over $\mathcal{R}_q^\ell$. Consequently,

$$(\mathbf{A}, \mathcal{R}_{\mathbf{d},\Delta}(\mathbf{u})) \equiv (\mathbf{A}, \mathcal{R}_{\mathbf{d},0}(\mathbf{u}))$$

as distributions, up to the natural relabeling of the uniform variable.

Proof. The map $\mathbf{u} \mapsto \mathbf{u} + \Delta$ is a bijection on the finite additive group $\mathcal{R}_q^\ell$. Therefore it preserves the uniform distribution. $\square$

Remark A.7 (Why the naive error-coupling argument is not used). The argument below does not rely on a statistical-distance bound between $\text{Compress}_{\mathbf{q}\mathbf{d}}(\mathbf{A}\mathbf{s} + \mathbf{e} + \Delta)$ and $\text{Compress}_{\mathbf{q}\mathbf{d}}(\mathbf{A}\mathbf{s} + \Delta)$. Such a bound is generally not small for practical LWE error distributions, because the error $\mathbf{e}$ may move coefficients across rounding boundaries with non-negligible probability. Instead, the shifted proof follows the LWE-to-LWR proof [32] path: statistical closeness is established before the final rounding step, and the public shift plus rounding is applied only as deterministic post-processing.

A.3 Main Reduction Theorem

Theorem A.8 (Shifted decisional LWE-to-LWR reduction). *Let n, k, ℓ, q, p be positive integers. Let $R = \mathcal{O}_K$ be the ring of integers of a number field K of degree n . Let χ_s be a distribution over $(R_q^\vee)^k$ (the secret space), and let χ_e be a β -bounded distribution over $K_{\mathbb{R}}^\ell$ (the error space), i.e., for every coefficient in the canonical embedding, $\|e_i\|_\infty \leq \beta$ with overwhelming probability. Let $\mathbf{d} = (d_1, \dots, d_\ell)$ be a product compression profile, and let B be a rounding basis of $R^\vee$ satisfying the admissibility conditions of [32] (specifically, $\gcd(q, [\mathcal{O}_K : \mathbb{Z}[\alpha]]) = 1$ and the entropy condition below). Let $\Delta \in \mathcal{R}_q^\ell$ be a fixed public offset.*

Assume that the following conditions hold.

1. **Lossy-matrix switching.** *For every $m \leq \ell$, there exists a lossy distribution Lossy_m over $\mathcal{R}_q^{m \times k}$ (Definition A.1) such that $\mathbf{A}' \leftarrow \$\mathcal{R}_q^{m \times k}$ and $\tilde{\mathbf{A}}' \leftarrow \text{Lossy}_m$ are computationally indistinguishable under the corresponding $\text{MLWE}_{k,m,\chi_s,\chi_e,q}$ assumption. We denote the loss of each such switch by ε_{sw} .*
2. **Entropy preservation in the lossy hybrids.** *For each row index $j \in [\ell]$, consider the lossy state $\tilde{\mathbf{st}}_j = (\tilde{\mathbf{A}}_{-j}, \{c_i\}_{i < j}, \{c_i\}_{i > j}, \Delta)$, where*

$$c_i = \begin{cases} \text{Compress}_q(u_i + \Delta_i, d_i), & i < j, \\ \text{Compress}_q(\langle \tilde{\mathbf{a}}_i, \mathbf{s} \rangle + \Delta_i, d_i), & i > j, \end{cases}$$

each $u_i \leftarrow \$\mathcal{R}_q$ is uniform and independent, and $\tilde{\mathbf{A}}_{-j}$ is sampled from the lossy distribution. For every ideal factor $\mathbf{q} \in \mathcal{Q}$ (where $\mathcal{Q}$ is the family of relevant ideal factors of qR from Theorem 5.7 of [32]), the conditional distribution of $S_{\mathbf{q}} := \mathbf{s} \bmod \mathbf{q}R^\vee$ satisfies the entropy requirement needed by the algebraic leftover hash lemma with error ε_{lh} . Concretely, in the min-entropy formulation, we assume

$$H_\infty(S_{\mathbf{q}} \mid \tilde{\mathbf{st}}_j) \geq \tau_{\text{ALHL}}(\varepsilon_{\text{lh}}) \quad \text{for all } j \in [\ell] \text{ and all } \mathbf{q} \in \mathcal{Q},$$

where $\tau_{\text{ALHL}}(\varepsilon_{\text{lh}})$ is the entropy threshold from Corollary 5.9 (or Corollary 5.10 for cyclotomic fields, or Corollary 5.11 for large ideal factors) of [32].

3. **Algebraic leftover-hash extraction.** *Under the entropy condition above, for a fresh uniform row $\mathbf{a} \leftarrow \$\mathcal{R}_q^k$, independent of $\tilde{\mathbf{st}}_j$, and for $z = \langle \mathbf{a}, \mathbf{s} \rangle \in \mathcal{R}_q$, one has*

$$\text{SD}((\tilde{\mathbf{st}}_j, \mathbf{a}, z), (\tilde{\mathbf{st}}_j, \mathbf{a}, u)) \leq \varepsilon_{\text{lh}},$$

where $u \leftarrow \$\mathcal{R}_q$ is uniform and independent of $(\tilde{\mathbf{st}}_j, \mathbf{a})$.

Then for every adversary $\mathcal{D}_\Delta$ against Shifted-MLWR $_{k,\ell,\chi_s,\mathbf{d},\Delta}$, there exists an adversary $\mathcal{B}_\Delta$ against the standard decisional MLWE $_{k,\ell,\chi_s,\chi_e,q}$ problem (i.e., distinguishing $(\mathbf{A}, \mathbf{A}\mathbf{s} + \mathbf{e})$ from $(\mathbf{A}, \mathbf{u})$, where $\mathbf{A} \leftarrow \$\mathcal{R}_q^{\ell \times k}$, $\mathbf{s} \leftarrow \chi_s^k$, $\mathbf{e} \leftarrow \chi_e^\ell$, and $\mathbf{u} \leftarrow \$\mathcal{R}_q^\ell$) such that

$$\boxed{\text{Adv}_{k,\ell,\chi_s,\mathbf{d},\Delta}^{\text{Shifted-MLWR}}(\mathcal{D}_\Delta) \leq \text{Adv}_{k,\ell,\chi_s,\chi_e,q}^{\text{MLWE}}(\mathcal{B}_\Delta) + \varepsilon_{\text{LW}}}$$

where

$$\varepsilon_{\text{LW}} := \ell \cdot \varepsilon_{\text{lh}} + \varepsilon_{\text{sw}}$$

and ε_{sw} is the total computational distinguishing advantage accumulated over the ℓ lossy-matrix switches (each bounded by the MLWE advantage of the corresponding reduction). In particular, ε_{lh} is the statistical distance from the algebraic leftover hash lemma (Theorem 5.7 of [32]), which depends only on $(n, q, \mathbf{d}, \mathcal{Q})$ and the entropy threshold τ_{ALHL} .

Proof. We prove the theorem by explicitly repeating the Liu–Wang decisional Module LWE-to-LWR hybrid argument [32], with the final rounding map replaced by the publicly shifted rounding map $\mathcal{R}_{\mathbf{d}, \Delta}(\mathbf{x}) = \text{Compress}_{\mathbf{q}, \mathbf{d}}(\mathbf{x} + \Delta)$.

Let

$$\mathbf{A} = \begin{pmatrix} \mathbf{a}_1^T \\ \vdots \\ \mathbf{a}_\ell^T \end{pmatrix} \in \mathcal{R}_q^{\ell \times k}, \quad \mathbf{s} \leftarrow \chi_s^k.$$

For each row $j \in [\ell]$, define $z_j = \langle \mathbf{a}_j, \mathbf{s} \rangle \in \mathcal{R}_q$. The real shifted-MLWR distribution is

$$\mathbf{H}_0 = (\mathbf{A}, (\text{Compress}_{\mathbf{q}}(z_1 + \Delta_1, d_1), \dots, \text{Compress}_{\mathbf{q}}(z_\ell + \Delta_\ell, d_\ell))).$$

The target random distribution is

$$\mathbf{H}_\ell = (\mathbf{A}, (\text{Compress}_{\mathbf{q}}(u_1 + \Delta_1, d_1), \dots, \text{Compress}_{\mathbf{q}}(u_\ell + \Delta_\ell, d_\ell))),$$

where $u_1, \dots, u_\ell \leftarrow \mathcal{R}_q$ are independent uniform ring elements.

Hybrid sequence. For $j = 0, 1, \dots, \ell$, let $\mathbf{H}_j = (\mathbf{A}, \mathbf{c}^{(j)})$, where

$$c_i^{(j)} = \begin{cases} \text{Compress}_{\mathbf{q}}(u_i + \Delta_i, d_i), & i \leq j, \\ \text{Compress}_{\mathbf{q}}(z_i + \Delta_i, d_i), & i > j. \end{cases}$$

Here each $u_i \leftarrow \mathcal{R}_q$ is uniform and independent. Thus $\mathbf{H}_0$ is the real shifted-MLWR distribution, and $\mathbf{H}_\ell$ is the random shifted-MLWR distribution.

Bounding consecutive hybrids. Fix a row index $j \in [\ell]$. Write $\mathbf{A}_{-j}$ for the matrix obtained from $\mathbf{A}$ by deleting row j , and let st_j contain:

- the public matrix rows $\mathbf{A}_{-j}$;
- the already replaced rounded values $\text{Compress}_{\mathbf{q}}(u_i + \Delta_i, d_i)$ for $i < j$;
- the not-yet-replaced rounded values $\text{Compress}_{\mathbf{q}}(z_i + \Delta_i, d_i)$ for $i > j$;
- all public parameters, including the shift Δ .

The only difference between $\mathbf{H}_{j-1}$ and $\mathbf{H}_j$ is the j -th rounded value: $\text{Compress}_{\mathbf{q}}(z_j + \Delta_j, d_j)$ versus $\text{Compress}_{\mathbf{q}}(u_j + \Delta_j, d_j)$.

Step 1: Switch the surrounding matrix to a lossy matrix. Before applying the leftover hash lemma to the fresh row $\mathbf{a}_j$, we switch $\mathbf{A}_{-j}$ to a lossy matrix $\tilde{\mathbf{A}}_{-j} \leftarrow \text{Lossy}$ (Definition A.1). This switch concerns only the distribution of the matrix rows in the state st_j and is independent of whether the final map applied to z_j is $x \mapsto \text{Compress}_{\mathbf{q}}(x, d_j)$ or $x \mapsto \text{Compress}_{\mathbf{q}}(x + \Delta_j, d_j)$. Therefore the same MLWE reduction used by Liu–Wang gives

$$\left| \Pr[\mathcal{D}_\Delta(\mathbf{H}_{j-1}) = 1] - \Pr[\mathcal{D}_\Delta(\tilde{\mathbf{H}}_{j-1}) = 1] \right| \leq \varepsilon_{\text{sw}},$$

where $\tilde{\mathbf{H}}_{j-1}$ denotes the hybrid with $\mathbf{A}_{-j}$ replaced by $\tilde{\mathbf{A}}_{-j}$. The same bound holds for $\tilde{\mathbf{H}}_j$.

Step 2: Entropy preservation under the lossy state with public shift. We now prove that the entropy condition required by the algebraic leftover hash lemma (Lemma A.2) remains valid in the shifted hybrid. This is the only part that requires a careful adaptation of the original Liu–Wang argument.

Fix any relevant ideal factor $\mathfrak{q} \mid qR$. Let $S_{\mathfrak{q}} := \mathbf{s} \bmod \mathfrak{q}R^{\vee}$. The lossy state for the j -th replacement can be written as

$$\tilde{\mathbf{st}}_j = \left(\tilde{\mathbf{A}}_{-j}, \{c_i\}_{i < j}, \{c_i\}_{i > j}, \Delta \right),$$

where

$$c_i = \begin{cases} \text{Compress}_{\mathfrak{q}}(u_i + \Delta_i, d_i), & i < j, \\ \text{Compress}_{\mathfrak{q}}(\langle \tilde{\mathbf{a}}_i, \mathbf{s} \rangle + \Delta_i, d_i), & i > j. \end{cases}$$

The components Δ and $\{c_i\}_{i < j}$ are independent of $\mathbf{s}$, and hence do not decrease the conditional min-entropy of $S_{\mathfrak{q}}$. The only potentially secret-dependent components are the future shifted-rounded values $\{c_i\}_{i > j}$.

For each $i > j$, conditioned on the corresponding lossy row $\tilde{\mathbf{a}}_i$, the value c_i is obtained by applying the deterministic map

$$f_{i, \Delta_i}(\mathbf{s}) = \text{Compress}_{\mathfrak{q}}(\langle \tilde{\mathbf{a}}_i, \mathbf{s} \rangle + \Delta_i, d_i).$$

Its image is contained in $(\mathbb{Z}_{2^{d_i}})^n$, and therefore $|\text{Im}(f_{i, \Delta_i})| \leq 2^{nd_i}$. By the standard min-entropy leakage bound, for any random variable X and any deterministic function f , $H_{\infty}(X \mid f(X)) \geq H_{\infty}(X) - \log |\text{Im}(f)|$. Applying this bound successively to the future shifted-rounded values gives

$$H_{\infty}(S_{\mathfrak{q}} \mid \tilde{\mathbf{A}}_{-j}, \{c_i\}_{i < j}, \{c_i\}_{i > j}, \Delta) \geq H_{\infty}(S_{\mathfrak{q}}) - \sum_{i > j} nd_i.$$

This is the same worst-case entropy loss as in the unshifted Liu–Wang hybrid, since the map $x \mapsto \text{Compress}_{\mathfrak{q}}(x + \Delta_i, d_i)$ has the same output alphabet size as $x \mapsto \text{Compress}_{\mathfrak{q}}(x, d_i)$.

However, the above argument only gives an a priori bound that may be loose. In the Liu–Wang proof, a much sharper entropy preservation is established by analyzing the “bad event” that many coefficients land on rounding borders. We now show that this analysis carries over verbatim to the shifted setting.

Define the shifted rounding border for a single coefficient as

$$\text{border}_{p, q}^{\Delta}(\tau) = \{x \in \mathbb{Z}_q : \exists y \in \mathbb{Z}, |y| \leq \tau, \lfloor x + \Delta \rfloor_p \neq \lfloor x + y + \Delta \rfloor_p\}.$$

Since translation by Δ is a bijection on $\mathbb{Z}_q$, we have $|\text{border}_{p, q}^{\Delta}(\tau)| = |\text{border}_{p, q}(\tau)| \leq 2\tau p$. Let $\tilde{\mathbf{A}}_{-j} = \mathbf{D}\mathbf{C} + \mathbf{F}$ be its lossy decomposition (as in Definition A.1). For each future row $i > j$, write $\tilde{\mathbf{a}}_i = \mathbf{d}_i\mathbf{C} + \mathbf{f}_i$ (with appropriate dimensions). For each coefficient index $t \in [n]$, define the bad event

$$I_{i, t} = \left[(\mathbf{d}_i\mathbf{C}\mathbf{s})_t \in \text{border}_{p, q}^{\Delta_i[t]}(|(\mathbf{f}_i\mathbf{s})_t|) \right].$$

Let $I = \sum_{i > j, t} I_{i, t}$. By the exact same argument as in Liu–Wang (Lemma 5.21 and the surrounding text), using the fact that $\mathbf{d}_i\mathbf{C}\mathbf{s}$ is uniform over $\mathcal{R}_q$ when $\mathbf{C}\mathbf{s}$ is invertible (and the invertibility failure is bounded by δ), we obtain

$$\Pr[|I| > \lambda] \leq \delta + \frac{n \log q}{2^{\lambda}} + 2^{-\lambda}.$$

The key point is that the border size and the uniformity of $\mathbf{d}_i\mathbf{C}\mathbf{s}$ are unaffected by the public shift Δ . Hence the same bound holds for the shifted bad event.

Given $|I| \leq \lambda$, the entire vector of shifted-rounded values $\{c_i\}_{i > j}$ can be reconstructed from $\mathbf{D}, \mathbf{C}, \mathbf{F}, \mathbf{C}\mathbf{s}$, and at most λ additional ring elements (the rounded values at the bad

positions). Each such additional element lies in a set of size at most p^n , so the total leakage is at most $\lambda n \log p$ bits. Therefore, by the smooth min-entropy lemma (Lemma 2.5 in [32]), for $\epsilon = \delta + \frac{n \log q}{2^\lambda} + 2^{-\lambda}$, we have

$$H_\infty^\epsilon(S_q \mid \tilde{\mathbf{st}}_j) \geq H_\infty(S_q) - (n + \lambda) \log q.$$

This is exactly the same entropy preservation bound as in the unshifted case. Thus, whenever the unshifted Liu–Wang theorem guarantees the required entropy threshold, the shifted state satisfies it as well.

Step 3: Apply the algebraic leftover hash lemma before rounding. Let $\mathbf{a}_j \leftarrow \$\mathcal{R}_q^k$ be the fresh j -th row, independent of the lossy state, and define $z_j = \langle \mathbf{a}_j, \mathbf{s} \rangle$. By the algebraic leftover hash lemma (Lemma A.2) under the entropy condition from Step 2,

$$\text{SD}((\tilde{\mathbf{st}}_j, \mathbf{a}_j, z_j), (\tilde{\mathbf{st}}_j, \mathbf{a}_j, u_j)) \leq \epsilon_{\text{lh}},$$

where $u_j \leftarrow \$\mathcal{R}_q$ is uniform and independent.

Now define the deterministic function

$$F_{j,\Delta_j}(\mathbf{st}, \mathbf{a}, x) = (\mathbf{st}, \mathbf{a}, \text{Compress}_q(x + \Delta_j, d_j)),$$

which acts only on the third argument x while leaving $(\mathbf{st}, \mathbf{a})$ unchanged. Since $(\tilde{\mathbf{st}}_j, \mathbf{a}_j)$ has the same marginal distribution in both terms above, applying F_{j,Δ_j} and invoking the data-processing inequality (Lemma A.5) gives

$$\text{SD}((\tilde{\mathbf{st}}_j, \mathbf{a}_j, \text{Compress}_q(z_j + \Delta_j, d_j)), (\tilde{\mathbf{st}}_j, \mathbf{a}_j, \text{Compress}_q(u_j + \Delta_j, d_j))) \leq \epsilon_{\text{lh}}.$$

Therefore,

$$\left| \Pr[\mathcal{D}_\Delta(\tilde{\mathbf{H}}_{j-1}) = 1] - \Pr[\mathcal{D}_\Delta(\tilde{\mathbf{H}}_j) = 1] \right| \leq \epsilon_{\text{lh}}.$$

Step 4: Switch the lossy matrix back to uniform. We switch $\tilde{\mathbf{A}}_{-j}$ back to a uniform matrix. This step is necessary to restore the uniform-matrix structure of $\mathbf{H}_j$, so that the next hybrid $\mathbf{H}_{j+1}$ starts from the same uniform-matrix distribution. As in Step 1, this switch is justified by the same MLWE assumption and is independent of the shifted rounding map. Hence the same switching loss ϵ_{sw} applies.

Combining Steps 1–4, for the j -th row replacement we obtain

$$|\Pr[\mathcal{D}_\Delta(\mathbf{H}_{j-1}) = 1] - \Pr[\mathcal{D}_\Delta(\mathbf{H}_j) = 1]| \leq 2\epsilon_{\text{sw}} + \epsilon_{\text{lh}}.$$

Absorbing the two switching losses into the advantage of the MLWE adversary constructed for this row gives

$$|\Pr[\mathcal{D}_\Delta(\mathbf{H}_{j-1}) = 1] - \Pr[\mathcal{D}_\Delta(\mathbf{H}_j) = 1]| \leq \text{Adv}_j^{\text{MLWE}}(\mathcal{B}_{\Delta,j}) + \epsilon_{\text{lh}}.$$

Step 5: Sum over all rows. By the triangle inequality over $\mathbf{H}_0, \mathbf{H}_1, \dots, \mathbf{H}_\ell$,

$$|\Pr[\mathcal{D}_\Delta(\mathbf{H}_0) = 1] - \Pr[\mathcal{D}_\Delta(\mathbf{H}_\ell) = 1]| \leq \sum_{j=1}^{\ell} \text{Adv}_j^{\text{MLWE}}(\mathcal{B}_{\Delta,j}) + \ell \cdot \epsilon_{\text{lh}} \leq \text{Adv}_{k,\ell,\chi_s,\chi_e,q}^{\text{MLWE}}(\mathcal{B}_\Delta) + \ell \cdot \epsilon_{\text{lh}}.$$

Identifying the endpoints. By construction, H_0 is exactly the real distribution of $\text{Shifted-MLWR}_{k,\ell,\chi_s,\mathbf{d},\Delta}$. For H_ℓ , since translation by the fixed public Δ is a bijection on $\mathcal{R}_q^\ell$, the vector $\mathbf{u} + \Delta$ is uniform over $\mathcal{R}_q^\ell$, so H_ℓ is exactly the random distribution of $\text{Shifted-MLWR}_{k,\ell,\chi_s,\mathbf{d},\Delta}$.

Thus,

$$\text{Adv}_{k,\ell,\chi_s,\mathbf{d},\Delta}^{\text{Shifted-MLWR}}(\mathcal{D}_\Delta) \leq \text{Adv}_{k,\ell,\chi_s,\chi_e,q}^{\text{MLWE}}(\mathcal{B}_\Delta) + \ell \cdot \varepsilon_{\text{lh}} + \varepsilon_{\text{sw}}.$$

Setting $\varepsilon_{\text{LW}} := \ell \cdot \varepsilon_{\text{lh}} + \varepsilon_{\text{sw}}$ yields the claimed bound. $\square$

Corollary A.9 (Ciphertext specialization). *Let*

$$\mathbf{d}_{\text{ct}} = (\underbrace{d_u, \dots, d_u}_k, d_v)$$

be the ciphertext compression profile. For a message $\mu_b \in \mathcal{M}$ let $M_b = \text{MsgEncode}(\mu_b) \in \mathcal{R}_q$ and define the fixed offset

$$\Delta_b = \begin{pmatrix} \mathbf{0} \\ M_b \end{pmatrix} \in \mathcal{R}_q^{k+1}.$$

Here M_b denotes the output of MsgEncode itself, which already lies in $\mathcal{R}_q$; no further decomposition is applied.

Assume that the hypotheses of Theorem A.8 hold for $\ell = k + 1$ and for the product compression profile $\mathbf{d}_{\text{ct}} = (d_u, \dots, d_u, d_v)$. In particular, assume that the corresponding ciphertext lossy hybrids satisfy the algebraic leftover-hash entropy condition for every relevant ideal factor $\mathfrak{q} \in \mathcal{Q}$ with error parameter $\varepsilon_{\text{lh},\text{ct}}$, and that the total Liu–Wang loss for this profile is $\varepsilon_{\text{LW},\text{ct}} = \varepsilon_{\text{LW}}(k, k+1, \chi_s, \chi_e, \mathbf{d}_{\text{ct}}, B)$. Then, for every fixed message $\mu_b \in \mathcal{M}$ and every adversary $\mathcal{D}$ against $\text{Shifted-MLWR}_{k,k+1,\chi_s,\mathbf{d}_{\text{ct}},\Delta_b}$, there exists an adversary $\mathcal{B}$ against $\text{MLWE}_{k,k+1,\chi_s,\chi_e,q}$ such that

$$\text{Adv}_{k,k+1,\chi_s,\mathbf{d}_{\text{ct}},\Delta_b}^{\text{Shifted-MLWR}}(\mathcal{D}) \leq \text{Adv}_{k,k+1,\chi_s,\chi_e,q}^{\text{MLWE}}(\mathcal{B}) + \varepsilon_{\text{LW},\text{ct}}.$$

Moreover, the bound is uniform over all messages $\mu_b \in \mathcal{M}$ because the proof loss depends only on the compression profile $\mathbf{d}_{\text{ct}}$ and on the Liu–Wang admissibility parameters, and not on the value of the fixed offset Δ_b .

Proof. This is an instantiation of Theorem A.8 with $\ell = k + 1$, $\mathbf{d} = \mathbf{d}_{\text{ct}} = (d_u, \dots, d_u, d_v)$, and $\Delta = \Delta_b = (\mathbf{0}, M_b)^T$. For each fixed message μ_b , the value $M_b = \text{MsgEncode}(\mu_b)$ is fixed, and hence Δ_b is a fixed offset in the corresponding shifted-MLWR instance.

The entropy condition required by the algebraic leftover hash lemma is the same as in the unshifted ciphertext profile. Indeed, the shifted rounding map appears only in the last ciphertext coordinate: $x \mapsto \text{Compress}_{\mathfrak{q}}(x + M_b, d_v)$, whose image is contained in $(\mathbb{Z}_{2d_v})^n$, the same output alphabet as $x \mapsto \text{Compress}_{\mathfrak{q}}(x, d_v)$. The first k coordinates use the ordinary compression parameter d_u . Thus the worst-case leakage from the ciphertext rounded values is bounded by $k \cdot nd_u + nd_v$, exactly as in the unshifted Liu–Wang ciphertext profile. Since the offset Δ_b is fixed and independent of the secret, conditioning on it does not reduce the conditional min-entropy of $\mathbf{s} \bmod \mathfrak{q}R^\vee$ for any relevant ideal factor $\mathfrak{q} \in \mathcal{Q}$.

Therefore all hypotheses of Theorem A.8 are satisfied for this ciphertext profile, and the stated advantage bound follows. Finally, in the random branch, if $\mathbf{u} \leftarrow \$\mathcal{R}_q^{k+1}$, then $\mathbf{u} + \Delta_b$ is also uniform over $\mathcal{R}_q^{k+1}$ by Lemma A.6. Hence the random branch is independent of the message-dependent offset, up to relabeling of the uniform variable. This proves the claim. $\square$

A.4 Scope and Justification of the Shifted-MLWR Assumption

The reduction established in Theorem A.8 and Corollary A.9 should be interpreted as a *conditional shift-stability result* for the Liu–Wang LWE-to-LWR framework. It demonstrates that the introduction of a fixed public offset Δ before rounding does not degrade the statistical or algebraic steps of the original proof.

We do not claim that the concrete parameter sets of WEAVER can be rigorously reduced to standard MLWE using the sufficient conditions of the Liu–Wang framework. Indeed, like almost all practical rounding-based schemes (including Saber [19] and Saber-like variants such as Smaug [15]), WEAVER’s parameters push the limits beyond where the sufficient entropy thresholds of the Algebraic Leftover Hash Lemma (ALHL) can be tightly satisfied.

Instead, we justify the soundness of the Shifted-MLWR assumption via the *stability of the proof path*. Theorem A.8 mathematically supports this stability by showing that the public shift Δ is fully compatible with the standard LWE-to-LWR hybrid argument. Because the statistical indistinguishability between the real and lossy hybrids is established *before* the final rounding step, the public translation followed by rounding acts merely as a deterministic post-processing. By the data-processing inequality, this deterministic step preserves the statistical distance. Consequently, the public shift introduces no hidden statistical leakage or algebraic vulnerability, guaranteeing that the Shifted-MLWR problem is as hard as its unshifted counterpart within the Liu–Wang framework, with no additional reduction loss.